Build & Break a Mini Secure Channel with Blockchain-based Decentralized Authentication

Εργασία στο μάθημα της Κρυπτογραφίας

ΣΤΥΛΙΑΝΟΣ ΒΑΣΙΛΕΙΟΥ
mai260004
Καθηγήτρια: Σοφία Πετρίδου
MsC Εφαρμοσμένης Πληροφορικής
Ιανουάριος 2026

Πίνακας Περιεχομένων

1. Εισαγωγή ... 1
   1.1 Κίνητρο της Εργασίας: Secure Channels και Αποκεντρωμένη Αυθεντικοποίηση ... 1
   1.2 Συνοπτική περιγραφή της εργασίας ... 1
   1.3 Δομή της εργασίας ... 2
2. Περιγραφή Συστήματος, Φάσεων και Απαιτήσεων ... 2
   2.1 Απαιτήσεις του Συστήματος ... 2
   2.2 Υψηλού Επιπέδου Επισκόπηση του Συστήματος ... 3
   2.3 Συνοπτική Επισκόπηση των Έξι Φάσεων ... 4
   2.4 Βασικό Υπόβαθρο Κρυπτογραφίας και Blockchain ... 5
3. Σύνολο Στοιχείων Ασφαλείας και Μοντέλο Απειλών ... 6
   3.1 Στοιχεία και Στόχοι Ασφαλείας ... 6
   3.2 Μοντέλα Αντιπάλων ... 6
   3.3 Μοντέλο Απειλών ... 7
4. Παρουσίαση των Φάσεων του Συστήματος ... 7
   4.1 Επεξήγηση Λογικής του Σχεδιασμού του Συστήματος ... 8
   4.2 Φάση 1 - Μη αυθεντικοποιημένος Diffie-Hellman ... 9
   4.3 Φάση 2 - Επίθεση Man-in-the-Middle ... 10
   4.4 Φάση 3 - Αυθεντικοποιημένος Diffie-Hellman με Ed25519 ... 11
   4.5 Φάση 4 – AEAD Secure Channel (ChaCha20-Poly1305) ... 12
   4.6 Φάση 5 – Αποκεντρωμένο Μητρώο Κλειδιών Βασισμένο στο Solana ... 13
   4.7 Φάση 6 - Σενάρια επιθέσεων στο blockchain ... 14
5. Υλοποίηση Συστήματος και Επεξήγηση Κώδικα ... 16
   5.1 Η δομή του Project και το Τεχνολογικό Stack ... 16
   5.2 Backend (Flask) και Οργάνωση των Φάσεων ... 17
   5.3 Το Κρυπτογραφικό Μέρος του Συστήματος (Python Modules) ... 18
      5.3.1 Φάση 1 – Ανταλλαγή κλειδιών Diffie-Hellman ... 18
      5.3.2 Φάση 2 – Επίθεση Man-in-the-Middle ... 19
      5.3.3 Φάση 3 – Αυθεντικοποιημένοι Συμμετέχοντες και Υπογεγραμμένο Handshake ... 20
      5.3.4 Φάση 4 – AEAD Secure Channel με ChaCha20-Poly1305 ... 23
   5.4 Solana / Anchor Program ... 26
      5.4.1 Python Client - Μητρώο Κλειδιών στο Solana ... 26
      5.4.2 Smart contract ... 30

5.4.3 Σενάρια Επιθέσεων στο Blockchain 31
6. Μαθήματα ασφαλείας, Συμπεράσματα και Μελλοντική Έρευνα 35
6.1 Συμπεράσματα για τον σχεδιασμό των Secure Channel 35
6.2 Συμπεράσματα για την αυθεντικοποίηση μέσω Blockchain 36
6.3 Μελλοντική Έρευνα 37
Βιβλιογραφία 38
Λέξεις Κλειδιά 39
Λίστα εικόνων 40
Λίστα Πινάκων 40
Τοποθεσία Κώδικα και Οδηγίες Εμφάνισης του UI 40

1. Εισαγωγή

1.1 Κίνητρο της Εργασίας: Secure Channels και Αποκεντρωμένη Αυθεντικοποίηση

Για την προστασία της εμπιστευτικότητας και της ακεραιότητας των δεδομένων, τα σύγχρονα συστήματα βασίζονται σε secure channels, ώστε δύο άγνωστες μεταξύ τους οντότητες να συμφωνούν σε ένα κοινό μυστικό, υπό ένα μη αξιόπιστο δίκτυο, στο οποίο υπάρχει πιθανός ενεργός επιτιθέμενος [1]. Παρότι υπάρχουν πρωτόκολλα όπως το TLS, τα οποία και καλύπτουν αυτή την ανάγκη σε περιβάλλοντα "παραγωγής" (production), συχνά αντιμετωπίζονται ως "μαύρα κουτιά" [2]. Με αυτό εννοούμε πως η υλοποίηση της αυθεντικοποίησης, της συμφωνίας κλειδιού και της προστασίας των μηνυμάτων, μένει αόρατη, όπως και το τι συμβαίνει όταν η αυθεντικοποίηση λείπει (π.χ. σε περίπτωση επίθεσης man-in-the-middle σε ένα μη αυθεντικοποιημένο Diffie-Hellman).

Ταυτόχρονα, οι κεντρικοποιημένες (centralized) υποδομές δημοσίων κλειδιών (PKI) και οι αρχές πιστοποίησης (certificate authorities) έχουν αναδειχθεί ως μοναδικά σημεία αστοχίας (single points of failure) για ολόκληρο το οικοσύστημα ασφάλειας [3] [4]. Για την αντιμετώπιση τέτοιου είδους προβλημάτων, προτείνεται η χρήση των blockchains ως αποκεντρωμένων πηγών εμπιστοσύνης, στα οποία τα δημόσια κλειδιά μπορούν να καταχωρούνται και να ελέγχονται χωρίς να υπάρχει οποιαδήποτε ανάγκη κάποιας κεντρικής αρχής [5]. Ένα παράδειγμα blockchain, το οποίο έχει χαμηλές καθυστερήσεις και μικρά fees, είναι το Solana, που είναι κατάλληλο για την υλοποίηση ενός αποκεντρωμένου μητρώου (registry) κλειδιών [6], [7].

Το κίνητρο της παρούσας εργασίας αντλείται από τη σύνδεση αυτών των δύο προβληματισμών. Η εργασία υλοποιεί βήμα προς βήμα ένα secure channel, επιδεικνύει πρακτικές επιθέσεις επάνω του και στη συνέχεια προσθέτει αποκεντρωμένη αυθεντικοποίηση μέσω ενός μητρώου κλειδιών στο Solana, ως εναλλακτική ή συμπληρωματική προσέγγιση στα παραδοσιακά PKI.

1.2 Συνοπτική περιγραφή της εργασίας

Η παρούσα εργασία υλοποιεί ένα secure channel 6 φάσεων, το οποίο ξεκινάει από έναν μη-αυθεντικοποιημένο Diffie-Hellman, ο οποίος είναι συγκεκριμένα ευάλωτος σε επιθέσεις man-in-the-middle, στη συνέχεια ενισχύεται προσθέτοντας αυθεντικοποιημένη ανταλλαγή κλειδιών, AEAD και ένα αποκεντρωμένο μητρώο κλειδιών που είναι βασισμένο στο Solana blockchain.

Τέλος παρουσιάζονται διάφορα σενάρια επιθέσεων στο blockchain και αναλύεται το πώς αποτρέπονται.

1.3 Δομή της εργασίας

Ακολουθεί συνοπτική περιγραφή των κεφαλαίων της εργασίας:

Κεφάλαιο 2: περιγράφει το σύστημα και τις απαιτήσεις του, παρουσιάζει τις έξι φάσεις και εισάγει το ελάχιστο αναγκαίο θεωρητικό υπόβαθρο σχετικά με τη κρυπτογραφία και το blockchain.

Κεφάλαιο 3: παρουσιάζει τα στοιχεία και τους στόχους ασφαλείας, καθώς και το μοντέλο απειλών.

Κεφάλαιο 4: επεξηγεί κάθε μία από τις έξι φάσεις σε υψηλό επίπεδο, με στόχο να γίνει σαφές το πώς λειτουργεί το σύστημα που υλοποιήθηκε σε κάθε στάδιο.

Κεφάλαιο 5: περιγράφει τον τρόπο υλοποίησης του συστήματος, την αρχιτεκτονική του και την οργάνωση του κώδικα, συμπεριλαμβανομένων του backend, του frontend και του Solana προγράμματος.

Κεφάλαιο 6: συγκεντρώνει τα βασικά συμπεράσματα όσον αφορά στην ασφάλεια, εξετάζει την επίδραση της αυθεντικοποίησης μέσω του blockchain και παρουσιάζει τις μελλοντικές προοπτικές.

2. Περιγραφή Συστήματος, Φάσεων και Απαιτήσεων

2.1 Απαιτήσεις του Συστήματος

Το σύστημα έχει σχεδιαστεί ώστε να χρησιμοποιείται ως εξής: ο χρήστης αφού ανοίξει το UI, εκτελεί την 1η φάση, για να δει μια λειτουργική αλλά μη αυθεντικοποιημένη ανταλλαγή κλειδιών και στη συνέχεια τρέχοντας διαδοχικά τις επόμενες φάσεις, να παρατηρεί το πώς εμφανίζονται οι επιθέσεις και το πώς σταδιακά αποτρέπονται.

Σχετικά με τις **τεχνικές απαιτήσεις**, το σύστημα πρέπει να:

- εγκαθιδρύει ένα κανάλι μεταξύ δύο οντοτήτων, της Alice και του Bob, σε ένα μη ασφαλές δίκτυο.

- υποστηρίζει έξι φάσεις στις οποίες η ασφάλεια ενισχύεται σε κάθε βήμα. Ξεκινάει από ένα μη αυθεντικοποιημένο Diffie Hellman και καταλήγει σε ένα AEAD κανάλι, το οποίο είναι βασισμένο σε blockchain.
- υλοποιεί μία επίθεση τύπου “man in the middle” στον μη αυθεντικοποιημένο Diffie Hellman.
- αλληλεπιδρά με ένα μητρώο κλειδιών βασισμένο σε Solana, με στόχο την καταχώριση και την επαλήθευση των δημοσίων κλειδιών.
- παρέχει ένα UI, το οποίο επιτρέπει στον χρήστη να ενεργοποιεί κάθε φάση, με στόχο να βλέπει καθαρά τα αντίστοιχα αποτελέσματα.

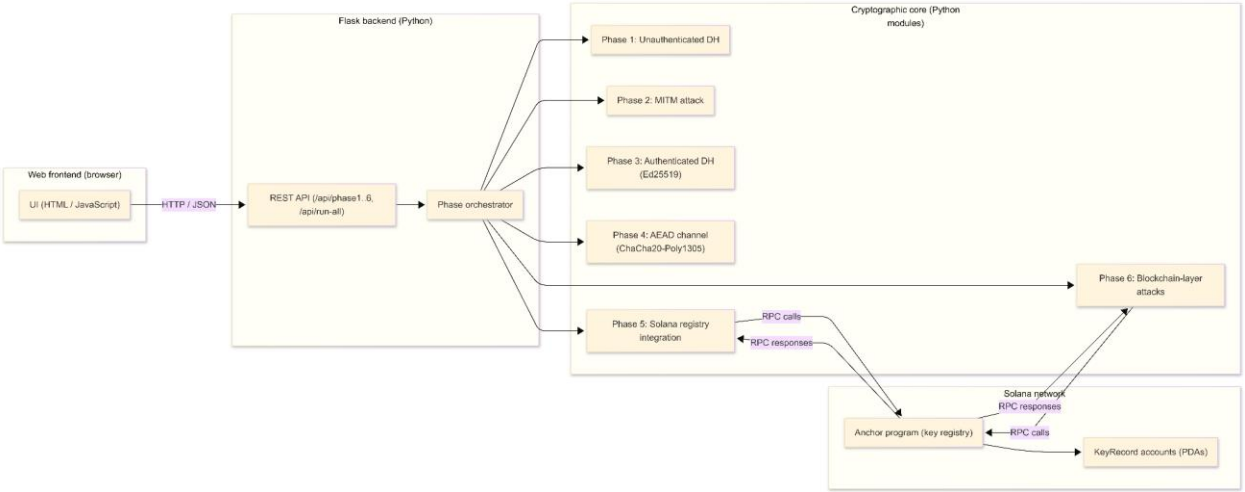
Οι κύριες **μη λειτουργικές απαιτήσεις** είναι οι εξής:

- **Διαφάνεια**: τα κλειδιά, τα αποτελέσματα της επαλήθευσης και οι εκβάσεις των επιθέσεων πρέπει να είναι όλα πλήρως ορατά και όχι κρυμμένα μέσα στις βιβλιοθήκες της αντίστοιχης γλώσσας προγραμματισμού.
- **Αναπαραγωγιμότητα**: με την εκτέλεση των ίδιων φάσεων και με τις ίδιες παραμέτρους, θα πρέπει να παράγονται τα ίδια αποτελέσματα.

2.2 Υψηλού Επιπέδου Επισκόπηση του Συστήματος

Για την υλοποίηση του συστήματος χρησιμοποιήθηκε Python για το backend, ένα απλό HTML/JavaScript για το frontend και ένα on-chain smart contract βασισμένο στο Solana blockchain.

Η ροή του συστήματος είναι η εξής: ο χρήστης κλικάρει για να τρέξει μία φάση στο frontend→ αποστέλλεται ένα αίτημα HTTP στο αντίστοιχο Flask endpoint (/api/phase1 … /api/phase6 και /api/run-all) → το endpoint καλεί το αντίστοιχο module της φάσης που επέλεξε να τρέξει ο χρήστης (και όπου απαιτείται τον Solana client) → επιστρέφεται ένα αποτέλεσμα σε JSON το οποίο στη συνέχεια παρουσιάζεται μετατετραμμένο στο UI.



Εικόνα 2.1: Αρχιτεκτονική υψηλού επιπέδου (frontend, Flask backend, modules των φάσεων, πρόγραμμα Solana).

2.3 Συνοπτική Επισκόπηση των Έξι Φάσεων

Το secure channel έχει σκόπιμα χωριστεί σε έξι φάσεις, με σκοπό κάθε μέρος της υλοποίησης να υπάρχει δυνατότητα να εισάγεται και ξεχωριστά:

1η Φάση: Βασική ανταλλαγή κλειδιών X25519 Diffie-Hellman μεταξύ δύο οντοτήτων (της Alice και του Bob) χωρίς καμία αυθεντικοποίηση.

2η Φάση: Προσομοίωση επίθεσης man-in-the-middle (από τον Mallory), η οποία έχει ως στόχο την υποκλοπή και αντικατάσταση των δημόσιων κλειδιών, με αποτέλεσμα τη δημιουργία δύο διαφορετικών κοινών μυστικών και την πλήρη παραβίαση του καναλιού.

3η Φάση: Αυθεντικοποίηση βασισμένη στο Ed25519, στην οποία κάθε μέρος έχει στην κατοχή της ένα κλειδί υπογραφής, υπογράφει τα “handshake” μηνύματα και επαληθεύει τις υπογραφές του άλλου μέρους, με αποτέλεσμα να αποτρέπεται έτσι η επίθεση man in the middle.

4η Φάση: Επέκταση της αυθεντικοποιημένης ανταλλαγής κλειδιών προσθέτοντας ένα ολοκληρωμένο AEAD secure channel το οποίο χρησιμοποιεί κλειδιά ChaCha20-Poly1305 και HKDF. Με αυτόν τον τρόπο το σύστημα διασφαλίζει την ακεραιότητα και την εμπιστευτικότητα των δεδομένων.

5η Φάση: Χρήση του Solana blockchain για την αποκεντρωμένη αποθήκευση των κλειδιών. Αυτό επιτυγχάνεται με τη δημιουργία ενός προγράμματος σε Anchor, το οποίο αποθηκεύει τα δημόσια κλειδιά ως program-derived accounts (PDAs) μέσω των διευθύνσεων των πορτοφολιών.

6η Φάση: Πραγματοποιείται προσομοίωση διαφόρων επιθέσεων σε επίπεδο blockchain (πχ. προσπάθειες καταχώρισης του κλειδιού του Mallory στη διεύθυνση της Alice) και αναδεικνύεται το πώς το blockchain είναι σχεδιασμένο ώστε να μπλοκάρει τέτοιες προσπάθειες πλαστοπροσωπίας.

2.4 Βασικό Υπόβαθρο Κρυπτογραφίας και Blockchain

Για την πλήρη κατανόηση της εργασίας, θα αναφερθούν οι βασικές έννοιες της κρυπτογραφίας και του blockchain. Τα κρυπτογραφικά δομικά στοιχεία και τα στοιχεία που χρησιμοποιούνται στο σύστημα παρουσιάζονται στον Πίνακα 2.1.

Κρυπτογραφικό στοιχείο	**Τύπος**	**Ρόλος στο σύστημα**
X25519	Αλγόριθμος ανταλλαγής κλειδιών	Παράγει ένα κοινό μυστικό μεταξύ της Alice και του Bob (Φάσεις 1-4) [8].
Ed25519	Σχήμα ψηφιακής υπογραφής	Υπογράφει και επαληθεύει τα μηνύματα του handshake (Φάση 3) [9].
HKDF	Συνάρτηση παραγωγής κλειδιών	Παράγει ένα ή περισσότερα συμμετρικά κλειδιά από το κοινό μυστικό της Diffie Hellman ανταλλαγής [10].
ChaCha20-Poly1305	Σχήμα AEAD	Παρέχει εμπιστευτικότητα και ακεραιότητα για τα δεδομένα (Φάση 4) [11] [12].
Solana Program Derived Addresses (PDAs)	Σχήμα διευθύνσεων blockchain	Δημιουργούν διευθύνσεις για λογαριασμούς KeyRecord, έτσι ώστε να ελέγχονται αποκλειστικά από το πρόγραμμα [13].
Anchor-based Solana program (KeyRecord)	On-chain smart contract	Ορίζει τη δομή του μητρώου των κλειδιών και τις εντολές για την καταχώριση και την ενημέρωσή τους (Φάσεις 5-6) [13].

Πίνακας 2.1: Βασικά στοιχεία της κρυπτογραφίας και του blockchain και ο ρόλος τους στο σύστημα.

Τα παραπάνω στοιχεία αποτελούν το τεχνικό υπόβαθρο πάνω στο οποίο στηρίζονται ο σχεδιασμός και η υλοποίηση της εργασίας.

3. Σύνολο Στοιχείων Ασφαλείας και Μοντέλο Απειλών

3.1 Στοιχεία και Στόχοι Ασφαλείας

Το σύστημα ασχολείται με ένα μικρό σύνολο στοιχείων (assets), τα οποία είναι σημαντικά για την ασφάλεια. Για κάθε ένα από αυτά χρειαζόμαστε κάποιες βασικές ιδιότητες [2].

Κατηγορία στοιχείου	**Ενδεικτικά στοιχεία**	**Ιδιότητες**
Σταθερό κλειδί	Μυστικά κλειδιά Ed25519, Solana wallet κλειδιά	Εμπιστευτικότητα, ακεραιότητα
Προσωρινό κλειδί	Ιδιωτικά κλειδιά X25519, AEAD κλειδιά ανά session	Εμπιστευτικότητα, μη επαναχρησιμοποίηση μεταξύ διαφορετικών session
Handshake δεδομένα	Δημόσια κλειδιά, υπογραφές, καταστάσεις πρωτοκόλλου	Ακεραιότητα, αυθεντικότητα, αντοχή σε τροποποιήσεις
Μηνύματα εφαρμογής	Κρυπτογραφημένα payloads	Εμπιστευτικότητα, ακεραιότητα
Εγγραφές ταυτότητας on-chain	KeyRecord PDAs, αποθηκευμένα δημόσια κλειδιά	Ακεραιότητα, ορθή δέσμευση wallet ↔ key, μη πλαστογραφησιμότητα

Πίνακας 3.1: Κύρια στοιχεία (assets) και ιδιότητες ασφαλείας

3.2 Μοντέλα Αντιπάλων

Τα μοντέλα των αντιπάλων που εξετάζουμε, είναι τα εξής:

Τύπος αντιπάλου	**Δυνατότητες**	**Παράδειγμα στην εργασία**
Παθητικός αντίπαλος δικτύου	Υποκλέπτει (Eavesdropping) και καταγράφει την κίνηση του καναλιού, χωρίς όμως να επεμβαίνει	Παρατηρεί την κίνηση της 1ης φάσης
Ενεργός αντίπαλος δικτύου	Υποκλέπτει (Eavesdropping), εισάγει, τροποποιεί και απορρίπτει μηνύματα (MITM)	Ο Mallory στη 2η φάση

Αντίπαλος στο επίπεδο του blockchain	Υποβάλλει αυθαίρετες συναλλαγές	Προσπαθεί να καταχωρίσει ή να τροποποιήσει κακόβουλα τις εγγραφές τύπου KeyRecord

Πίνακας 3.2: Τύποι αντιπάλων και οι δυνατότητές τους

3.3 Μοντέλο Απειλών

Με βάση τα παραπάνω στοιχεία ασφαλείας και τους αντιπάλους που αντιμετωπίζουμε, οι κύριες απειλές είναι οι εξής:

- **Σε επίπεδο δικτύου**
  - Υποκλοπή σε μη αυθεντικοποιημένο traffic (στις πρώτες φάσεις).
  - Επίθεση MITM στον Diffie-Hellman: αντικατάσταση των δημόσιων κλειδιών ώστε ο επιτιθέμενος να μπορεί να ελέγχει και τα δύο κοινά μυστικά (2η φάση) [1] [14].
  - Παρέμβαση σε ciphertexts: τροποποίηση των μηνυμάτων που είναι προστατευμένα με AEAD (φάση 4) [12] [11].
- **Σε επίπεδο ταυτότητας και αυθεντικοποίησης**
  - Πλαστοπροσωπία (spoofing) μακροπρόθεσμων κλειδιών, όταν δεν υπάρχει σωστή δέσμευση ταυτότητας-κλειδιού
  - Αντικατάσταση εφήμερων κλειδιών στη διαδικασία του handshake, όταν δεν υπάρχουν υπογραφές ή έλεγχοι μέσω κάποιου μητρώου [15].
- **Στο επίπεδο Blockchain**
  - Επιθέσεις λανθασμένης καταχώρισης (mis-registration): καταχώριση ενός κλειδιού, υπό την ταυτότητα ενός άλλου πορτοφολιού [16].
  - Μη εξουσιοδοτημένες ενημερώσεις των λογαριασμών τύπου KeyRecord.

4. Παρουσίαση των Φάσεων του Συστήματος

Στο κεφάλαιο αυτό αναλύεται η συμπεριφορά του συστήματος σε κάθε μία από τις φάσεις από την οπτική του χρήστη, ο οποίος αλληλεπιδρά με το web UI. Ο κύριος στόχος του κεφαλαίου αυτού είναι να αναδειχθεί τι συμβαίνει σε κάθε φάση, πώς αποτυπώνεται στο demo, και τι

συμπέρασμα προκύπτει απο κάθε βήμα, χωρίς όμως να αναλυθεί ο κώδικας (το οποίο συμβαίνει στο 5ο κεφάλαιο).

4.1 Επεξήγηση Λογικής του Σχεδιασμού του Συστήματος

Το σύστημα είναι συγκεκριμένα δομημένο ως μια αλληλουχία φάσεων, και όχι ως ένα ολοκληρωμένο secure channel. Κάθε μια φάση εισάγει κάθε φορά μία ακριβώς βελτίωση όσον αφορά στην ασφάλεια. Με αυτόν τον τρόπο ο χρήστης βιώνει τη συνεχόμενη βελτίωση του secure channel:

Ελαττωματικό → αποδεδειγμένα ευάλωτο → ασφαλές σε επίπεδο δικτύου → βελτίωση ασφάλειας με AEAD → ενσωματωμένο σε ένα αποκεντρωμένο μητρώο→ ασφαλές απέναντι σε επιθέσεις στο μητρώο

Φάση	**Κύρια αλλαγή**	**Ασφάλεια**
1	Απλή ανταλλαγή κλειδιών X25519	Η απλή συμφωνία κλειδιού δεν είναι ασφαλής
2	Επίθεση MITM	Ο μη αυθεντικοποιημένος DH είναι ευάλωτος σε MITM
3	Υπογραφές Ed25519 στο handshake	Η αυθεντικοποίηση του DH αποτρέπει την επίθεση MITM
4	AEAD (ChaCha20-Poly1305)	Με το AEAD ενισχύεται η εμπιστευτικότητα και η ακεραιότητα
5	Registry κλειδιών βασισμένο σε Solana PDAs	Οι ταυτότητες μπορούν να αποθηκευτούν στο blockchain
6	Σενάρια επιθέσεων στο μητρώο	Ένα σωστά σχεδιασμένο μητρώο αποτρέπει επιθέσεις

Πίνακας 4.1: Σύνοψη των φάσεων και παρουσίαση του βασικού «μαθήματος ασφαλείας» που μεταδίδει καθεμία από αυτές.

4.2 Φάση 1 - Μη αυθεντικοποιημένος Diffie-Hellman

Στην 1η φάση, όταν ο χρήστης κλικάρει το “Run Phase 1” στο UI, το backend παράγει ζεύγη κλειδιών X25519 για την Alice και τον Bob, εκτελεί μια βασική ανταλλαγή κλειδιών Diffie-Hellman και εμφανίζει τα σχετικά αποτελέσματα.

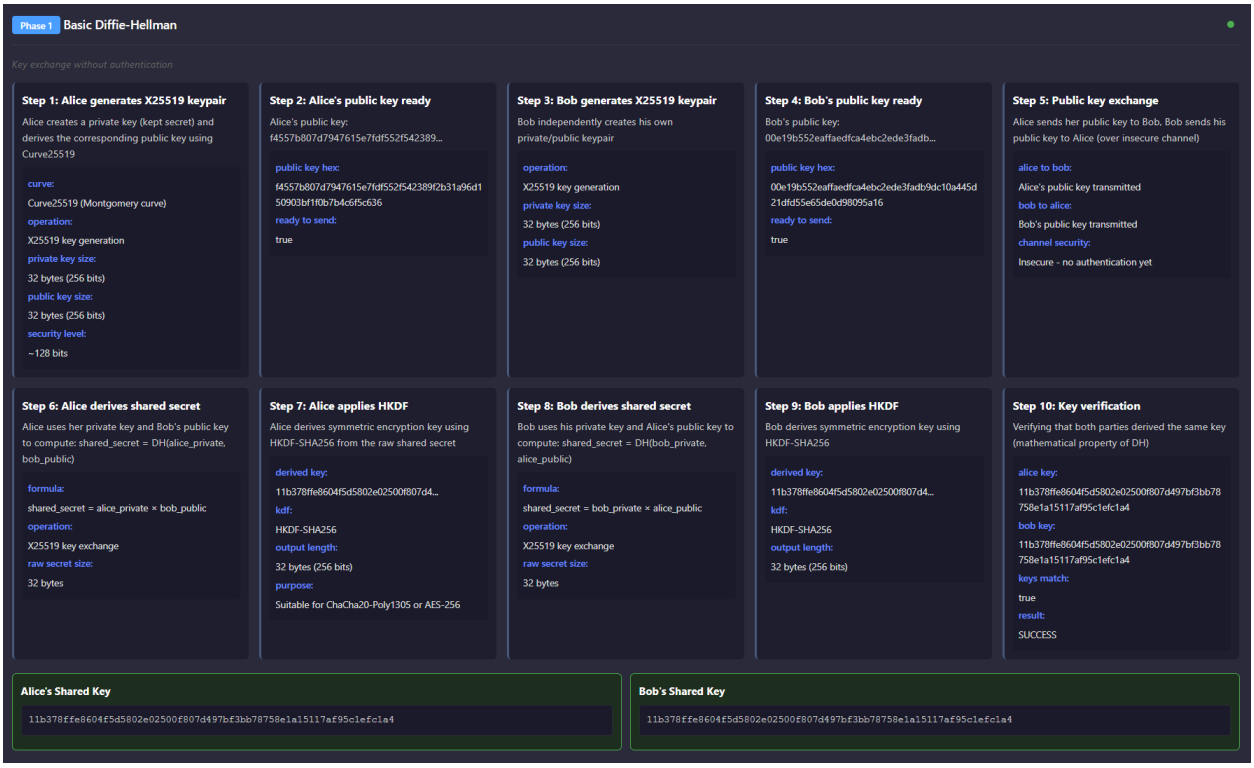
Η **1η φάση** αξιοποιεί την κρυπτογραφία για να:

1. παραγάγει ένα ζεύγος κλειδιών X25519 για την Alice.
2. παραγάγει ένα ζεύγος κλειδιών X25519 για τον Bob.
3. ανταλλάξει τα δημόσια κλειδιά μέσω του backend.
4. υπολογίσει το κοινό μυστικό και από τις δύο πλευρές και να το μορφοποιήσει ως ένα συμμετρικό κλειδί.

Στην φάση αυτή δεν υπάρχει κάποιο βήμα για αυθεντικοποίηση. Το σύστημα απλώς υποθέτει πως τα δημόσια κλειδιά που λαμβάνει κάθε μέρος είναι ορθά.

Συμπέρασμα

Η φάση 1 δείχνει πως ο Diffie-Hellman λειτουργεί επιτυχώς σε ένα «καλόηθες» περιβάλλον. Ταυτόχρονα όμως, θέτει τα θεμέλια του προβλήματος με το οποίο ασχολείται η εργασία, καθώς δεν υπάρχει καμία εγγύηση πως τα δημόσια κλειδιά ανήκουν πράγματι στον επιθυμητό συνομιλητή [8].



Εικόνα 4.1: Το UI της πρώτης Φάσης, με τα δημόσια κλειδιά της Alice και του Bob και ταυτιζόμενα συμμετρικά κλειδιά. Για πιο αναλυτική προβολή των logs, τρέξε το UI.

4.3 Φάση 2 - Επίθεση Man-in-the-Middle

Στην 2η φάση όταν ο χρήστης κλικάρει το “Run Phase 2” στο UI, εμφανίζονται 3 οντότητες: η Alice, ο Bob, και ο Mallory. Απο τα log που εμφανίζονται στο UI, ο χρήστης μπορεί να καταλάβει εύκολα πως ο Mallory υποκλέπτει και τροποποιεί το handshake.

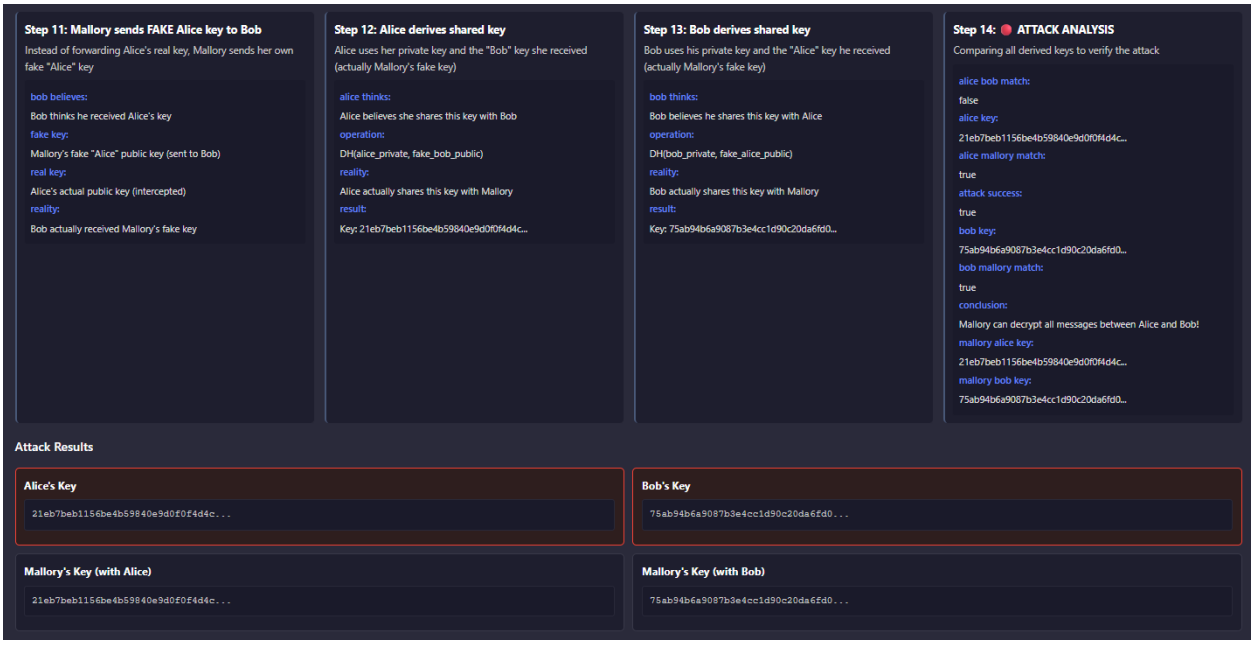
Όσον αφορά στην κρυπτογραφία, **η 2η φάση** επαναχρησιμοποιεί τον ίδιο X25519 μηχανισμό με την 1η φάση και επιπλέον:

1. Ο Mallory δημιουργεί το δικό του ζεύγος X25519 κλειδιών.
2. Κάθε φορά που η Alice στέλνει ένα δημόσιο κλειδί, ο Mallory το υποκλέπτει, παράγει ένα κοινό μυστικό με την Alice και στη συνέχεια στέλνει το δικό του δημόσιο κλειδί στον Bob.
3. Αντίστοιχα, ο Mallory υποκλέπτει το δημόσιο κλειδί του Bob, παράγει ένα κοινό μυστικό μαζί του και στέλνει το δικό του δημόσιο κλειδί (του Mallory) στην Alice.
4. Το σύστημα εμφανίζει και τα τρία κοινά κλειδιά.

Εφόσον εξακολουθεί να μην υπάρχει αυθεντικοποίηση στο σύστημα, τίποτα δεν εμποδίζει τον Mallory να παρουσιαστεί ως «Bob» στην Alice και ως «Alice» στον Bob [14].

Συμπέρασμα

Η 2η φάση δείχνει πως ο μη αυθεντικοποιημένος Diffie-Hellman είναι ανεπαρκής μπροστά σε έναν ενεργό επιτιθέμενο στο δίκτυο. Ο χρήστης βλέπει αναλυτικά το “σπάσιμο” του καναλιού βήμα προς βήμα, και όχι απλά μια θεωρητική επίθεση.



Εικόνα 4.2: Στιγμιότυπο μέρους της 2ης φάσης στο UI, η οποία απεικονίζει την επίθεση MITM και τη μη ταύτιση των κλειδιών που παράγονται από την Alice και τον Bob. Για πιο αναλυτική προβολή των logs, τρέξε το UI.

4.4 Φάση 3 - Αυθεντικοποιημένος Diffie-Hellman με Ed25519

Στην 3η Φάση, όταν ο χρήστης επιλέξει στο UI το “Run Phase 3”, το backend περιλαμβάνει πλέον και τα κλειδιά υπογραφής Ed25519 για την Alice και τον Bob.

Η **3η φάση** επεκτείνει το handshake της 1ης φάσης, προσθέτοντας υπογραφές. Πιο συγκεκριμένα:

1. Κάθε οντότητα δημιουργεί ένα ζεύγος κλειδιών υπογραφής Ed25519.
2. Η Alice υπογράφει το δημόσιο κλειδί X25519 με το μυστικό κλειδί Ed25519 και στέλνει το ζεύγος (public key, signature).
3. Ο Bob επαληθεύει την υπογραφή πριν αποδεχτεί το κλειδί, χρησιμοποιώντας το δημόσιο κλειδί Ed25519 της Alice.
4. Ο Bob απαντά με τον δικό του συνδυασμό (public key, signature), τον οποίο η Alice επαληθεύει με τον ίδιο τρόπο.

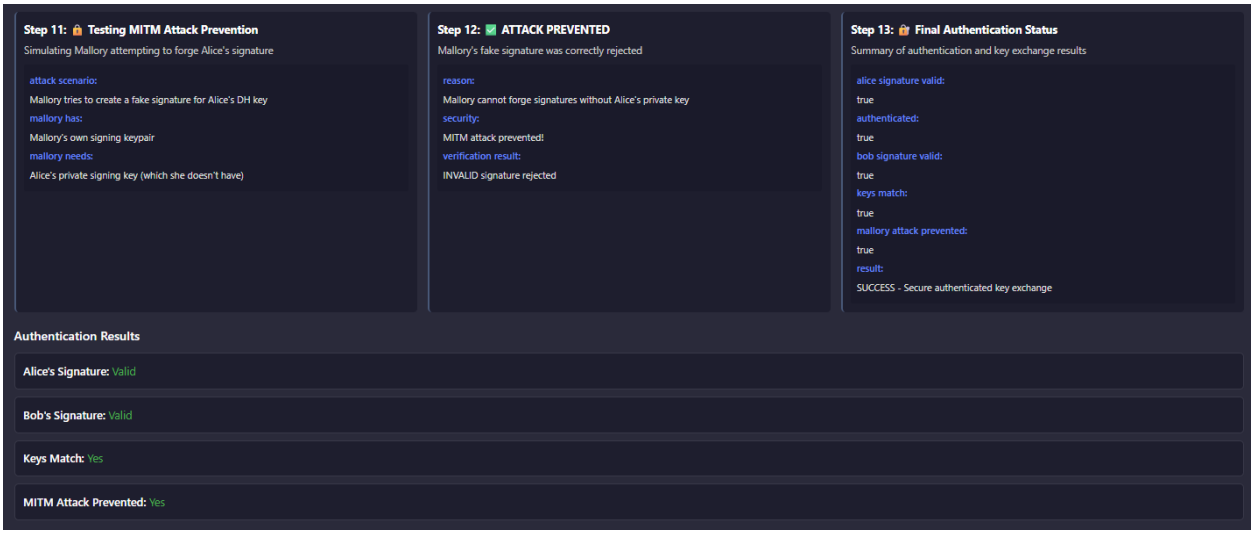
5. Μόνο αν και οι δύο επαληθεύσεις επιτύχουν, τότε προχωρούν στη δημιουργία του κοινού μυστικού [15], [17].

Ένας επιτιθέμενος τύπου MITM όπως είναι **ο Mallory** πρέπει είτε:

- να πλαστογραφήσει μια υπογραφή Ed25519 με το κλειδί της Alice ή του Bob (κάτι που δεν θεωρείται εφικτό) [18].
- να χρησιμοποιήσει το δικό του μακροπρόθεσμο κλειδί, γεγονός που τον αποκαλύπτει ως «Mallory» και όχι ως Alice ή Bob.

Συμπέρασμα

Η 3η φάση δείχνει πως μέσω της δέσμευσης του εφήμερου κλειδιού σε μακροπρόθεσμες ταυτότητες μέσω υπογραφών, το σύστημα μπλοκάρει τις πιθανές επιθέσεις τύπου MITM [18]. Ο χρήστης μπορεί μέσω του UI να συγκρίνει τα αποτελέσματα των φάσεων 2 και 3 καθώς και τις λεπτομέρειές τους, ώστε να εξαγάγει τα κατάλληλα συμπεράσματα.



Εικόνα 4.3: Στιγμιότυπο του UI για τη 3η φάση. Για πιο αναλυτική προβολή των logs, τρέξε το UI.

4.5 Φάση 4 – AEAD Secure Channel (ChaCha20-Poly1305)

Όταν ο χρήστης επιλέγει το “Run Phase 4”, η **4η φάση**:

1. παίρνει το αυθεντικοποιημένο κοινό μυστικό από την 3η φάση.
2. παράγει ένα AEAD κλειδί των 32-byte μέσω HKDF.
3. δημιουργεί ένα στιγμιότυπο ChaCha20-Poly1305 με αυτό το κλειδί και ένα νέο “nonce” των 12 bytes [11].

4. χρησιμοποιεί το AEAD API για να κρυπτογραφήσει και να αυθεντικοποιήσει το μήνυμα της Alice, και στην πλευρά του Bob να επαληθεύσει και να αποκρυπτογραφήσει το μήνυμα, απορρίπτοντας οποιοδήποτε τροποποιημένο ciphertext.

Συμπέρασμα

Η 4η φάση δείχνει πως δεδομένου ενός αυθεντικοποιημένου κλειδιού που προέρχεται από την 3η φάση, το AEAD παρέχει εμπιστευτικότητα και ακεραιότητα στα δεδομένα της εφαρμογής.

Εικόνα 4.4: Στιγμιότυπο του UI για τη 4η φάση 4. Για πιο αναλυτική προβολή των logs, τρέξε το UI.

4.6 Φάση 5 – Αποκεντρωμένο Μητρώο Κλειδιών Βασισμένο στο Solana

Όταν ο χρήστης εκτελεί τη **φάση 5** στο UI μέσω του “Run Phase 5”:

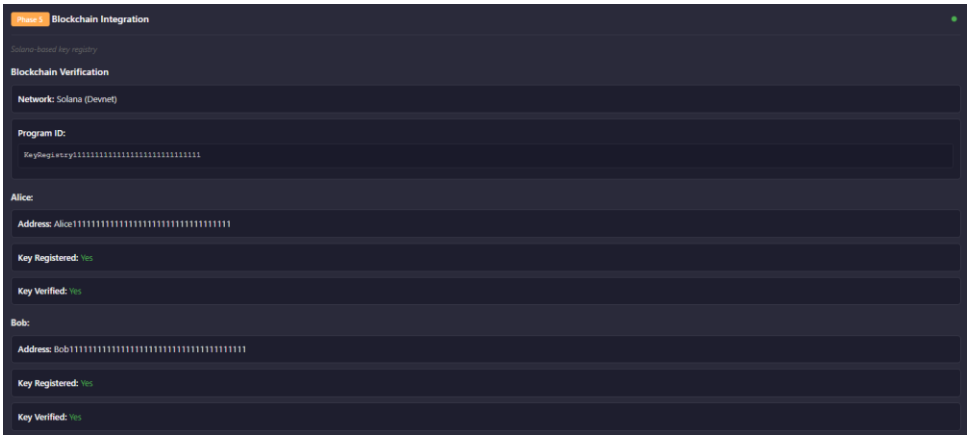
1. Υποθέτουμε ότι η Alice και ο Bob διαθέτουν διευθύνσεις πορτοφολιών στο Solana, οι οποίες λειτουργούν ως on-chain ταυτότητες.
2. Το backend καλεί έναν Solana client, ο οποίος επικοινωνεί με ένα smart contract, που είναι ειδικά σχεδιασμένο ως ένα μητρώο δημοσίων κλειδιών.
3. Για κάθε διεύθυνση δημιουργείται ή ενημερώνεται ένας λογαριασμός τύπου **KeyRecord**, ο οποίος:
   - συνδέεται με τη συγκεκριμένη διεύθυνση του ιδιοκτήτη
   - αποθηκεύει το αντίστοιχο δημόσιο κλειδί Ed25519 που χρησιμοποιείται στις φάσεις 3 και 4 ως ένα κλειδί ταυτότητας.
4. Το backend στέλνει τις συναλλαγές καταχώρισης των κλειδιών προς το smart contract και, αφού αυτές επιβεβαιωθούν, διαβάζει ξανά τα KeyRecord, ώστε να ελέγξει ότι:

- για τη διεύθυνση της Alice είναι καταχωρισμένο το “σωστό” δημόσιο κλειδί της Alice
- για τη διεύθυνση του Bob είναι καταχωρισμένο το “σωστό” δημόσιο κλειδί του Bob

5. Το αποτέλεσμα αυτής της διαδικασίας (αν δηλαδή το κλειδί είναι καταχωρισμένο και ταιριάζει) επιστρέφεται στο UI ως απλές ενδείξεις Key Registered: Yes/No και Key Verified: Yes/No για κάθε πλευρά.

Συμπέρασμα

Η 5η φάση δείχνει το πώς τα μακροπρόθεσμα κλειδιά ταυτότητας μπορούν να αποθηκεύονται σε ένα δημόσιο blockchain και το πώς ο έλεγχος ταυτότητας μετακινείται από τον τοπικό υπολογιστή σε μια κοινά προσβάσιμη, αποκεντρωμένη on-chain καταχώριση [6].



Εικόνα 4.5: Στιγμιότυπο του UI για τη φάση 5, με επιτυχημένη καταχώριση και επαλήθευση των κλειδιών της Alice και του Bob στο Solana blockchain. Για πιο αναλυτική προβολή των logs, τρέξε το UI.

4.7 Φάση 6 - Σενάρια επιθέσεων στο blockchain

Μόλις ο χρήστης ξεκινήσει την **6η φάση**, αφού “στηθεί” ξανά το registry και καταχωριστούν τα κλειδιά της Alice και του Bob, ο χρήστης βλέπει τέσσερα συγκεκριμένα σενάρια επίθεσης από τον Mallory. Πιο συγκεκριμένα:

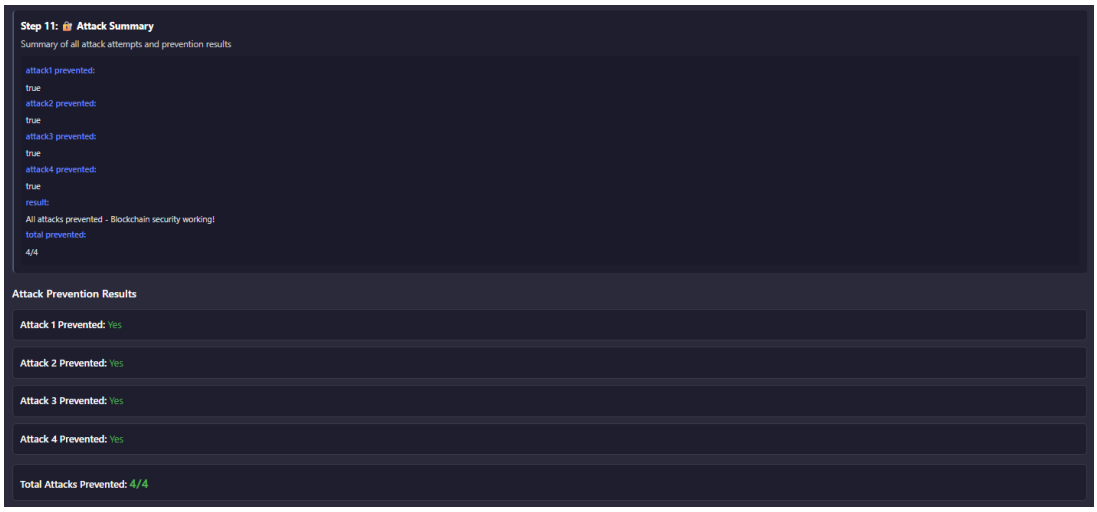
- Χρησιμοποιείται η ίδια λογική του μητρώου όπως με την 5η φάση. Για κάθε διεύθυνση υπάρχει ένας on-chain “φάκελος” (KeyRecord) ο οποίος συνδέει τη διεύθυνση του πορτοφολιού με ένα δημόσιο κλειδί.
- Ο **Mallory** δημιουργεί το δικό του πορτοφόλι και επιχειρεί 4 διαφορετικές επιθέσεις:

- ○ είτε καταχωρίζοντας το κλειδί της Alice για λογαριασμό της Alice, αλλά υπογράφοντας με το δικό του πορτοφόλι
- ○ είτε καταχωρίζοντας το δικό του κλειδί με τη διεύθυνση της Alice
- ○ είτε παρουσιάζοντας το κλειδί που δεν αντιστοιχεί στο κλειδί που έχει ήδη καταχωριστεί για μια διεύθυνση
- ○ είτε καταχωρίζοντας σωστά το δικό του κλειδί για τη δική του διεύθυνση και προσπαθώντας να το χρησιμοποιήσει σαν να ήταν της Alice

- Σε όλες τις περιπτώσεις όπου ο Mallory επιχειρεί να "πειράξει" τον λογαριασμό άλλης διεύθυνσης, το smart contract απορρίπτει τις ενέργειές του, επειδή ο υπογράφων της συναλλαγής δεν είναι ο νόμιμος ιδιοκτήτης της διεύθυνσης.
- Από την πλευρά του, το **backend** δεν εμπιστεύεται ό,τι του «λέει» ο Mallory. Κάθε φορά που πρέπει να ελέγξει την ταυτότητα της Alice:
  - ○ διαβάζει από το μητρώο ποιο κλειδί είναι καταχωρισμένο για τη διεύθυνση της Alice και αποδέχεται μόνο αυτό.
- Έτσι, οποιοσδήποτε συνδυασμός (διεύθυνση, κλειδί) που δεν ταιριάζει με τα on-chain δεδομένα απορρίπτεται.

Συμπέρασμα

Η φάση 6 αναδεικνύει το πώς μπορεί να μπλοκάρει το blockchain όλες τις βασικές μορφές πλαστοπροσωπίας που επιχειρεί ο Mallory.



Εικόνα 4.6: Στιγμιότυπο του UI για την 6η φάση. Για πιο αναλυτική προβολή των logs, τρέξε το UI.

5. Υλοποίηση Συστήματος και Επεξήγηση Κώδικα

Ο κύριος σκοπός του κεφαλαίου αυτού είναι η επεξήγηση του κώδικα που χρησιμοποιήθηκε για την υλοποίηση του secure channel. Ακολουθεί τη δομή της εργασίας και των φάσεών της, εστιάζοντας όμως σε συγκεκριμένα αρχεία, συναρτήσεις και βιβλιοθήκες που χρησιμοποιήθηκαν στο backend της Python, στα phase modules και στα Solana/Anchor components.

5.1 Η δομή του Project και το Τεχνολογικό Stack

Η υλοποίηση αποτελείται από μια μικρή αλλά πλήρη web εφαρμογή, με τη χρήση της Python για το backend, συγκεκριμένα modules για κάθε φάση, ένα απλό frontend και ένα Solana smart contract:

- **backend/app.py** - εφαρμογή Flask και χρήση του REST API για την κλήση όλων των φάσεων.
- **backend/phases/phase1_dh/dh_exchange.py** - μη αυθεντικοποιημένος X25519 Diffie-Hellman.
- **backend/phases/phase2_mitm/mallory_attack.py** - προσομοίωση επίθεσης MITM.
- **backend/phases/phase3_auth/authenticated_dh.py** - αυθεντικοποιημένος Diffie-Hellman με Ed25519.
- **backend/phases/phase4_aead/secure_channel.py** - AEAD secure channel με ChaCha20-Poly1305.
- **backend/phases/phase5_solana/solana_registry_client.py** - Solana key-registry client.
- **backend/phases/phase6_blockchain_attack/blockchain_mitm_attack.py** - σενάρια επιθέσεων στο blockchain.
- **frontend/templates/index.html, frontend/static/main.js, frontend/static/styles.css** - UI.

Το backend έχει υλοποιηθεί με την χρήση **Python**, χρησιμοποιώντας:

- Flask για HTTP routing.

- τη βιβλιοθήκη της Python “cryptography” για τα X25519, Ed25519, HKDF και ChaCha20-Poly1305.
- εργαλεία τύπου solana / anchorpy για την παραγωγή PDAs στο Devnet του Solana.

Τα κρυπτογραφικά imports που χρειάζονται και επαναχρησιμοποιούνται σε όλες τις φάσεις, δηλώνονται στο backend/app.py:

```
from phases.phase1_dh.dh_exchange import generate_x25519_keypair, public_bytes, derive_shared_key
from cryptography.hazmat.primitives.asymmetric import ed25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305
from cryptography.exceptions import InvalidSignature, InvalidTag
import binascii
import struct
import secrets
```

5.2 Backend (Flask) και Οργάνωση των Φάσεων

Το Flask backend λειτουργεί ως ένας “ενορχηστρωτής”, καθώς για κάθε φάση εκθέτει ένα HTTP endpoint και αναθέτει όλη τη λογική των φάσεων στα αντίστοιχα modules.

Το κύριο αρχείο **backend/app.py** δημιουργεί την εφαρμογή Flask και ορίζει τα **REST API**:

- POST /api/phase1 → run_phase1()
- POST /api/phase2 → run_phase2()
- POST /api/phase3 → run_phase3()
- POST /api/phase4 → run_phase4()
- POST /api/phase5 → run_phase5()
- POST /api/phase6 → run_phase6()
- POST /api/run-all → run_all_phases() (τρέχει όλες τις φάσεις διαδοχικά)

Κάθε ένα **route**:

1. λαμβάνει ένα request,
2. καλεί την αντίστοιχη συνάρτηση φάσης στο backend,
3. επιστρέφει το παραγόμενο λεξικό (dictionary) Python ως JSON, με πεδία τα: success, phase, title, steps, data, visualization και summary.

Το μοτίβο που ακολουθείται για τους handlers των φάσεων στο app.py είναι το εξής:

```
@app.route('/api/phase1', methods=['POST'])
```

```
def run_phase1():
    try:
```

5.3 Το Κρυπτογραφικό Μέρος του Συστήματος (Python Modules)

Οι κύριες κρυπτογραφικές λειτουργίες του συστήματος είναι υλοποιημένες με τη βιβλιοθήκη “cryptography” στις φάσεις 1-4, με κάθε μία φάση να εισάγει και ένα νέο χαρακτηριστικό ασφαλείας.

5.3.1 Φάση 1 – Ανταλλαγή κλειδιών Diffie-Hellman (phase1_dh/dh_exchange.py)

Η 1η φάση υλοποιείται στο αρχείο backend/phases/phase1_dh/dh_exchange.py και παρέχει συναρτήσεις για τις ακόλουθες λειτουργίες:

- τη δημιουργία ενός ζεύγους κλειδιών X25519 για μία οντότητα - generate_x25519_keypair() [8]

```
def generate_x25519_keypair():
    private = x25519.X25519PrivateKey.generate()
    public = private.public_key()
    return private, public
```

- τη σειριοποίηση ενός δημόσιου κλειδιού για την εμφάνιση και τη μεταφορά του - public_bytes(public_key)
- την παραγωγή ενός κοινού κλειδιού χρησιμοποιώντας X25519 και HKDF - derive_shared_key(our_private: x25519.X25519PrivateKey, their_public_bytes: bytes, info: bytes = b”secure_channel_v1")

```
def derive_shared_key(our_private: x25519.X25519PrivateKey, their_public_bytes: bytes, info:
bytes = b"secure_channel_v1"):
    their_public = x25519.X25519PublicKey.from_public_bytes(their_public_bytes)
    shared_secret = our_private.exchange(their_public)
    hkdf = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=info,
    )
    key = hkdf.derive(shared_secret)
    return key
```

Το endpoint της 1ης φάσης καλεί τις συναρτήσεις αυτές για την Alice και τον Bob, παράγει τα δημόσια κλειδιά τους σε hex και παράγει σε κάθε πλευρά ένα συμμετρικό κλειδί μέσω HKDF.

5.3.2 Φάση 2 – Επίθεση Man-in-the-Middle (phase2_mitm/mallory_attack.py)

Η 2η φάση υλοποιείται στο αρχείο backend/phases/phase2_mitm/mallory_attack.py, περιέχει τις ίδιες συναρτήσεις με την 1η φάση και επιπλέον παρέχει:

Την κλάση **Participant** η οποία αναπαριστά οντότητες όπως την Alice ή τον Bob. Περιέχει:

- μια μέθοδο generate_keypair(), η οποία δημιουργεί ένα ζεύγος κλειδιών X25519 χρησιμοποιώντας τη μέθοδο generate_x25519_keypair() της πρώτης φάσης και αποθηκεύει τα bytes του δημόσιου κλειδιού (μέσω της μεθόδου public_bytes(...))
- μια μέθοδο derive_key(peer_public_bytes), η οποία καλεί την derive_shared_key(...) για να υπολογίσει ένα συμμετρικό κλειδί από το δημόσιο κλειδί του peer και το αποθηκεύει στη μεταβλητή self.shared_key, χωρίς να πραγματοποιεί καμία αυθεντικοποίηση σχετικά με το ποιος είναι πραγματικά ο peer.

Η κλάση **Mallory** αναπαριστά τον επιτιθέμενο. Περιέχει:

- δύο ζεύγη X25519 κλειδιών (alice_priv/alice_pub και bob_priv/bob_pub), ώστε να μπορεί ο επιτιθέμενος να προσποιείται την Alice στον Bob και τον Bob στην Alice
- Την μέθοδο intercept_and_replace(...) η οποία:
    - ελέγχει αν το μήνυμα που “εκλάπη” προέρχεται από την Alice ή τον Bob
    - χρησιμοποιεί το κατάλληλο ιδιωτικό κλειδί για να καλέσει τη μέθοδο derive_shared_key(...) και να υπολογίσει το self.key_with_alice ή το self.key_with_bob,
    - επιστρέφει τα δικά της public key bytes (πχ. bob_pub_bytes όταν υποδύεται τον Bob και alice_pub_bytes όταν υποδύεται την Alice), αντί για το πραγματικό δημόσιο κλειδί του αποστολέα

```
    def intercept_and_replace(self, message_from, message_to):
        real_pub_bytes = message_from.public_key_bytes
        if message_from.name == "Alice":
            self.key_with_alice = derive_shared_key(
                self.bob_priv,
                real_pub_bytes
            )
            print(f"{self.name}: [INTERCEPTED] Alice's public key")
            print(f"{self.name}: Derived key with Alice (hex):
{binascii.hexlify(self.key_with_alice).decode()}")
            print(f"{self.name}: [FORWARDING] Fake Bob's public key to Alice")
```

```
            return self.bob_pub_bytes
        elif message_from.name == "Bob":
            self.key_with_bob = derive_shared_key(
                self.alice_priv,
                real_pub_bytes
            )
            print(f"{self.name}: [INTERCEPTED] Bob's public key")
            print(f"{self.name}: Derived key with Bob (hex): {binascii.hexlify(self.key_with_bob).decode()}")
            print(f"{self.name}: [FORWARDING] Fake Alice's public key to Bob")
            return self.alice_pub_bytes
```

- μια βοηθητική μέθοδο, όπως είναι η can_decrypt(encrypted_message, sender), η οποία απλώς ελέγχει αν έχει ήδη εγκαθιδρυθεί το αντίστοιχο shared key με την Alice ή τον Bob (self.key_with_alice / self.key_with_bob).

Η συνάρτηση **demonstrate_mitm_attack()** στη συνέχεια:

1. δημιουργεί τα αντικείμενα alice = Participant("Alice"), bob = Participant("Bob") και mallory = Mallory(),
2. καλεί την alice.generate_keypair() και την bob.generate_keypair(),
3. προσομοιώνει την ανταλλαγή των δημόσιων κλειδιών δρομολογώντας και τις δύο κατευθύνσεις μέσω της συνάρτησης mallory.intercept_and_replace(...), ώστε η Alice και ο Bob να μην δουν ποτέ το πραγματικό δημόσιο κλειδί ο ένας του άλλου
4. καλεί την alice.derive_key(fake_bob_key_for_alice) και bob.derive_key(fake_alice_key_for_bob) χρησιμοποιώντας τα κλειδιά που επιστρέφει ο Mallory,
5. υπολογίζει και εμφανίζει τα τέσσερα κλειδιά (alice.shared_key, bob.shared_key, mallory.key_with_alice, mallory.key_with_bob) και ελέγχει οτι:
    - alice.shared_key == mallory.key_with_alice,
    - bob.shared_key == mallory.key_with_bob,
    - alice.shared_key != bob.shared_key.

5.3.3 Φάση 3 – Αυθεντικοποιημένοι Συμμετέχοντες και Υπογεγραμμένο Handshake (phase3_auth/authenticated_dh.py)

Η 3η φάση υλοποιείται στο αρχείο backend/phases/phase3_auth/authenticated_dh.py. Διορθώνει την ευπάθεια της φάσης 2, εισάγοντας αυθεντικοποιημένους συμμετέχοντες που

συνδυάζουν την ανταλλαγή κλειδιών X25519 με τις υπογραφές Ed25519. Το authenticated_dh.py εισάγει τις βιβλιοθήκες των x25519 και ed25519 από το cryptography.hazmat.primitives.asymmetric, των HKDF και SHA-256 από τα cryptography.hazmat.primitives.kdf.hkdf και cryptography.hazmat.primitives.hashes, και χρησιμοποιεί την InvalidSignature από τη βιβλιοθήκη cryptography.exceptions για τον εντοπισμό των αποτυχημένων ελέγχων υπογραφής.

Αρχικά ορίζονται οι βοηθητικές συναρτήσεις generate_x25519_keypair(), public_bytes() και derive_shared_key(...) οι οποίες υλοποιήθηκαν στην 1η φάση και στη συνέχεια εισάγει την κλάση **AuthenticatedParticipant,** στην οποία:

- Κάθε **AuthenticatedParticipant** (δηλαδή η Alice ή ο Bob) διατηρεί:
    - ένα ζεύγος X25519 κλειδιών (self.dh_private, self.dh_public, self.dh_public_bytes) για τον Diffie-Hellman,
    - ένα ζεύγος Ed25519 κλειδιών υπογραφής (self.signing_private, self.signing_public, self.signing_public_bytes) το οποίο χρησιμοποιείται ως το μακροπρόθεσμο κλειδί ταυτότητας,
    - ένα πεδίο self.shared_key για το τελικό συμμετρικό κλειδί.
- Η μέθοδος **generate_keypairs():**
    - Καλεί την generate_x25519_keypair() και την public_bytes(...) για να ορίσει το self.dh_private και το self.dh_public_bytes
    - δημιουργεί ένα ζεύγος κλειδιών Ed25519 με την ed25519.Ed25519PrivateKey.generate()
    - παράγει το δημόσιο κλειδί Ed25519 με την self.signing_private.public_key(),
    - το σειριοποιεί σε raw bytes με την self.signing_public.public_bytes(…).
- Η μέθοδος **sign_dh_public_key()**:
    - υπολογίζει μια υπογραφή Ed25519 πάνω στα self.dh_public_bytes χρησιμοποιώντας τη συνάρτηση self.signing_private.sign(self.dh_public_bytes),
    - επιστρέφει την υπογραφή, ώστε να σταλεί μαζί με το δημόσιο κλειδί DH.

```
def sign_dh_public_key(self):
    signature = self.signing_private.sign(self.dh_public_bytes)
    print(f"{self.name}: Signed DH public key")
```

```
        print(f"{self.name}: Signature (hex): {binascii.hexlify(signature).decode()}")
        return signature
```

- Η μέθοδος **verify_and_derive_key(peer_dh_pub_bytes, peer_signing_pub_bytes, signature):**
    1. ανακατασκευάζει το δημόσιο κλειδί Ed25519 του peer με την ed25519.Ed25519PublicKey.from_public_bytes(peer_signing_pub_bytes)
    2. Καλεί την peer_signing_pub.verify(signature, peer_dh_pub_bytes) και αν η επαλήθευση αποτύχει, γίνεται “raise” εξαίρεση InvalidSignature και η μέθοδος επιστρέφει (False, None) χωρίς να παραχθεί κάποιο κλειδί [18],
    3. αν η επαλήθευση πετύχει, παράγει το κοινό κλειδί μέσω της derive_shared_key(self.dh_private, peer_dh_pub_bytes), το αποθηκεύει στο self.shared_key και επιστρέφει (True, self.shared_key).

Επιπλέον, η κλάση **AuthenticatedMallory** αναπαριστά έναν επιτιθέμενο που προσπαθεί να επαναλάβει την επίθεση MITM της 2ης φάσης, αυτή τη φορά όμως με αυθεντικοποίηση:

- Η κλάση **AuthenticatedMallory** δημιουργεί τα δικά της ζεύγη κλειδιών X25519 και Ed25519 στον constructor, αποθηκεύοντας τα self.dh_pub_bytes και self.signing_pub_bytes.
- Η μέθοδος **intercept_and_replace(message_from)**:
    - υπογράφει το δικό της δημόσιο κλειδί DH με self.signing_priv.sign(self.dh_pub_bytes),
    - επιστρέφει το τριπλό (self.dh_pub_bytes, self.signing_pub_bytes, signature) ως ένα πλαστό υπογεγραμμένο μήνυμα.

```
    def intercept_and_replace(self, message_from):
        signature = self.signing_priv.sign(self.dh_pub_bytes)
        print(f"{self.name}: [ATTEMPTING] Intercepting {message_from.name}'s message")
        print(f"{self.name}: [CREATING] Fake signed message with own keys (signature won't
match!)")
        return self.dh_pub_bytes, self.signing_pub_bytes, signature
```

Η συνάρτηση **demonstrate_authenticated_exchange()** στη συνέχεια εκτελεί το αυθεντικοποιημένο handshake και την απόπειρα επίθεσης:

1. Δημιουργεί τα αντικείμενα alice = AuthenticatedParticipant("Alice") και bob = AuthenticatedParticipant("Bob").

2. Καλεί την alice.generate_keypairs(), alice.sign_dh_public_key() και στη συνέχεια την bob.verify_and_derive_key(alice.dh_public_bytes, alice.signing_public_bytes, alice_signature). Αν το bob_valid είναι False, τότε το πρωτόκολλο τερματίζεται.
3. Καλεί την bob.generate_keypairs(), bob.sign_dh_public_key() και έπειτα την alice.verify_and_derive_key(bob.dh_public_bytes, bob.signing_public_bytes, bob_signature). Αν το alice_valid είναι False, τότε το πρωτόκολλο τερματίζεται.
4. Συγκρίνει τα alice.shared_key και bob.shared_key. Αν η σύγκριση επιστρέφει true, τότε εμφανίζεται: "[SUCCESS] Alice and Bob derived the same shared key".

Το **δεύτερο μέρος** της **demonstrate_authenticated_exchange()** ελέγχει την ανθεκτικότητα του πρωτοκόλλου σε επίθεση MITM:

5. Δημιουργεί το αντικείμενο mallory = AuthenticatedMallory().
6. Καλεί την mallory.intercept_and_replace(alice) και λαμβάνει έναν πλαστό τριπλό συνδυασμό (fake_alice_dh, fake_alice_signing, fake_sig).
7. Ανακατασκευάζει το γνήσιο δημόσιο κλειδί Ed25519 της Alice από τα alice.signing_public_bytes με την ed25519.Ed25519PublicKey.from_public_bytes(alice.signing_public_bytes) και επιχειρεί να εκτελέσει τη συνάρτηση alice_signing_pub.verify(fake_sig, fake_alice_dh).
8. Η επαλήθευση αυτή κάνει “raise” το σφάλμα InvalidSignature και εμφανίζεται: "[SUCCESS] ATTACK PREVENTED: Bob correctly rejected Mallory's fake signature".

5.3.4 Φάση 4 – AEAD Secure Channel με ChaCha20-Poly1305 (phase4_aead/secure_channel.py)

Η 4η φάση υλοποιείται στο backend/phases/phase4_aead/secure_channel.py. Βασίζεται πάνω στην αυθεντικοποιημένη ανταλλαγή κλειδιών Diffie-Hellman της 3ης φάσης και συνεχίζει προσθέτοντας πλήρη AEAD κρυπτογράφηση με ChaCha20-Poly1305, ώστε το σύστημα να μεταμορφωθεί σε ένα πλήρες secure channel. Το αρχείο πραγματοποιεί imports των x25519 και ed25519 από το cryptography.hazmat.primitives.asymmetric, των HKDF και hashes από τα cryptography.hazmat.primitives.kdf.hkdf και cryptography.hazmat.primitives, της ChaCha20Poly1305 από το cryptography.hazmat.primitives.ciphers.aead, καθώς και των InvalidSignature και InvalidTag από το cryptography.exceptions.

Όπως και στις φάσεις 1 και 3, ορίζονται και χρησιμοποιούνται οι βοηθητικές συναρτήσεις generate_x25519_keypair(), public_bytes() και derive_shared_key(...) με ακριβώς τον ίδιο τρόπο όπως περιγράφηκε προηγουμένως, καθώς και επαναχρησιμοποιείται από τη φάση 3 η αφηρημένη κλάση AuthenticatedParticipant, για την αυθεντικοποιημένη ανταλλαγή κλειδιών.

Η 4η φάση εισάγει την κλάση SecureChannel η οποία πραγματοποιεί το αυθεντικοποιημένο hanshake με AEAD.

Κάθε στιγμιότυπο της **SecureChannel** (είτε για την Alice είτε για τον Bob) παρέχει:

- ένα αντικείμενο AuthenticatedParticipant στο self.participant για την αυθεντικοποιημένη ανταλλαγή DH (όπως και στη 3η φάση)
- ένα cipher AEAD αντικείμενο στο self.cipher, το οποίο αρχικοποιείται μετά από επιτυχή αυθεντικοποίηση
- μετρητές self.nonce_counter και self.peer_nonce_counter οι οποίοι μετρούν τα εξερχόμενα και τα εισερχόμενα nonces.

Η μέθοδος **establish_channel(peer_dh_pub_bytes, peer_signing_pub_bytes, peer_signature):**

- καλεί την self.participant.verify_and_derive_key(peer_dh_pub_bytes, peer_signing_pub_bytes, peer_signature) ακριβώς όπως και στη 3η φάση
- αν η επαλήθευση αποτύχει, επιστρέφεται False και δεν εγκαθίσταται κάποιο κανάλι
- αν η επαλήθευση επιτύχει, λαμβάνει το κοινό κλειδί των 32-byte και αρχικοποιεί το AEAD cipher και επιστρέφει True, υποδεικνύοντας ότι το secure channel είναι έτοιμο.

```
    def establish_channel(self, peer_dh_pub_bytes, peer_signing_pub_bytes, peer_signature):
        valid, shared_key = self.participant.verify_and_derive_key(
            peer_dh_pub_bytes,
            peer_signing_pub_bytes,
            peer_signature)
        if not valid:
            return False
        self.cipher = ChaCha20Poly1305(shared_key)
        print(f"{self.name}: Secure channel established. AEAD cipher initialized.")
        return True
```

Η μέθοδος **send_message(plaintext: bytes, associated_data: bytes = b""):**

- προϋποθέτει ότι το self.cipher έχει αρχικοποιηθεί (δηλαδή ότι το κανάλι έχει ξεκινήσει),

- κατασκευάζει ένα μοναδικό nonce των 12 bytes, συνδυάζοντας έναν counter των 64-bit σε μορφή big-endian με 4 τυχαία bytes [11], [19]:

```
nonce = struct.pack('>Q', self.nonce_counter) + os.urandom(4)
self.nonce_counter += 1
```

- καλεί την:

```
ciphertext = self.cipher.encrypt(nonce, plaintext, associated_data)
```

παράγοντας ένα ciphertext με ενσωματωμένο tag αυθεντικοποίησης Poly1305,

- επιστρέφει το ζεύγος των (ciphertext, nonce) για αποστολή.

Η μέθοδος **receive_message(ciphertext: bytes, nonce: bytes, associated_data: bytes = b"")**:

- προϋποθέτει επίσης ότι το self.cipher έχει ήδη αρχικοποιηθεί,
- προσπαθεί να αποκρυπτογραφήσει και να επαληθεύσει το μήνυμα με:

```
plaintext = self.cipher.decrypt(nonce, ciphertext, associated_data)
```

- αν η επαλήθευση επιτύχει, επιστρέφει το plaintext,
- αν η επαλήθευση αποτύχει (π.χ. επειδή το ciphertext ή το tag έχουν τροποποιηθεί), η ChaCha20Poly1305 κάνει “raise” InvalidTag. Η εξαίρεση αυτή πιάνεται (catch), καταγράφεται ως αλλοίωση (tampering) και ξαναγίνεται “raise”.

Η βοηθητική μέθοδος **get_public_keys()** καλεί την self.participant.sign_dh_public_key() και επιστρέφει το τριπλό (self.participant.dh_public_bytes, self.participant.signing_public_bytes, signature), ώστε και τα δύο SecureChannel instances να εκτελέσουν την ίδια αυθεντικοποιημένη διαδικασία ανταλλαγής όπως συμβαίνει και στην 3η φάση.

Η συνάρτηση **demonstrate_secure_channel()** είναι η συνάρτηση η οποία κάνει “επίδειξη” όλης της λειτουργικότητας της 4ης φάσης:

- Δημιουργεί τα αντικείμενα: alice_channel = SecureChannel("Alice") και bob_channel = SecureChannel("Bob").
- Εκτελεί την αυθεντικοποιημένη ανταλλαγή των κλειδιών μεταξύ της Alice και του Bob με το ίδιο μοτίβο που συμβαίνει και στην 3η φάση, με τη μόνη διαφορά να είναι η κλήση των alice_channel.get_public_keys() / bob_channel.get_public_keys() και του establish_channel(...) σε κάθε πλευρά.
- Όταν και τα δύο κανάλια αναφέρουν επιτυχία, η Alice στέλνει ένα σταθερό μήνυμα (πχ. "Hello Bob! This is a secret message.") μέσω του alice_channel.send_message(...), και ο

Bob το αποκρυπτογραφεί με το bob_channel.receive_message(...), ελέγχοντας ότι το plaintext ταυτίζεται.

- Στη συνέχεια, ο Bob στέλνει μια απάντηση μέσω του bob_channel.send_message(...) και η Alice την αποκρυπτογραφεί συμμετρικά με το alice_channel.receive_message(...).
- Τέλος, η συνάρτηση για να επιδειξει την προστασία που παρέχει ως προς την ακεραιότητα του μηνύματος, αλλοιώνει (κάνει flip) ένα bit σε ένα ήδη παραγόμενο ciphertext (π.χ. tampered_ciphertext[10] ^= 0xFF) καλώντας τη bob_channel.receive_message(bytes(tampered_ciphertext), nonce). Αυτό κάνει "raise"InvalidTag, και το script εμφανίζει μήνυμα επιτυχίας το οποίο δηλώνει πως η αλλοίωση (tampering) ανιχνεύθηκε και το μήνυμα απορρίφθηκε.

5.4 Solana / Anchor Program

Η 5η και η 6η φάση επεκτείνουν το secure channel με ένα μητρώο βασισμένο στο Solana, καθώς και με συγκεκριμένα σενάρια επιθέσεων στο blockchain. Η υλοποίηση των φάσεων αυτών βασίζεται κυρίως σε ένα Python client (solana_registry_client.py) και σε ένα smart contract γραμμένο σε Anchor (lib.rs).

5.4.1 Python Client - Μητρώο Κλειδιών στο Solana (phase5_solana/solana_registry_client.py)

Η ενσωμάτωση με το blockchain πραγματοποιείται από έναν Python client, ο οποίος υλοποιείται στο αρχείο backend/phases/phase5_solana/solana_registry_client.py. Χρησιμοποιεί τις κλάσεις solana.rpc.api.Client, solders.keypair.Keypair, solders.pubkey.Pubkey και το anchorpy (Program, Provider, Wallet, Idl) για να προσομοιώσει την αλληλεπίδραση με ένα μητρώο κλειδιών στο Solana (key-registry) και επίσης εισάγει την κλάση SecureChannel από την 4η φάση.

Η κεντρική κλάση του client είναι η S**olanaKeyRegistryClient**, η οποία ενσωματώνει την πρόσβαση RPC και την παραγωγή των PDAs. Κάθε στιγμιότυπό της διατηρεί:

- ένα RPC URL endpoint στο self.rpc_url
- μια σύνδεση solana_api.Client στο self.connection
- το αναγνωριστικό του προγράμματος στο self.program_id (προεπιλογή "KeyRegistry11111111111111111111111111111"),
- ένα απλοποιημένο Anchor IDL στο self.idl, το οποίο φορτώνεται μέσω της μεθόδου _load_idl().

Ο **κατασκευαστής** (contructor):

```
    def __init__(self, rpc_url: str = "https://api.devnet.solana.com", program_id: Optional[str]
= None):
        self.rpc_url = rpc_url
        self.connection = solana_api.Client(rpc_url)
        self.program_id = Pubkey.from_string(program_id or
"KeyRegistry11111111111111111111111111111")
        self.idl = self._load_idl()
        print(f"Solana Key Registry Client initialized")
        print(f"RPC URL: {rpc_url}")
        print(f"Program ID: {self.program_id}")
```

αρχικοποιεί τον client, εμφανίζει το επιλεγμένο RPC URL και το program ID.

Η μέθοδος **_load_idl()** επιστρέφει ένα in-memory IDL, το οποίο περιέχει τρεις εντολές (registerKey, updateKey, verifyKey) και έναν τύπο λογαριασμού (KeyRecord)

Η μέθοδος **_derive_key_record_pda(owner_pubkey: Pubkey)**:

- υπολογίζει για κάθε key record το Program Derived Address (PDA), χρησιμοποιώντας ως seeds το "key_record" και το δημόσιο κλειδί του owner
- καλεί την Pubkey.find_program_address(seeds, self.program_id) και επιστρέφει το αποτέλεσμα (pda, bump)

Η μέθοδος **register_key(wallet_keypair: Keypair, ed25519_public_key: bytes) -> str:**

- ελέγχει ότι το ed25519_public_key είναι ακριβώς 32 bytes
- υπολογίζει το owner_pubkey = wallet_keypair.pubkey() και το αντίστοιχο key_record_pda μέσω της μεθόδου _derive_key_record_pda(owner_pubkey)
- καταγράφει τον owner, την PDA και το δημόσιο κλειδί σε hex
- εμφανίζει ότι έχει προετοιμαστεί μια registration instruction (σε μία ρεαλιστική υλοποίηση, αυτή θα κατασκευαζόταν μέσω της Anchor και θα αποστελλόταν ως συναλλαγή)
- επιστρέφει μία υπογραφή συναλλαγής mock: "mock_tx_signature_for_demo".

Η μέθοδος **verify_key(owner_pubkey_str: str, ed25519_public_key: bytes) -> bool:**

- αναλύει τη διεύθυνση του owner με τη χρήση της Pubkey.from_string(owner_pubkey_str),
- παράγει την ίδια key_record_pda μέσω της _derive_key_record_pda(...),

- καλεί την self.connection.get_account_info(key_record_pda) για να ελέγξει αν υπάρχει λογαριασμός τύπου KeyRecord στη συγκεκριμένη PDA,
- εμφανίζει διαγνωστικά μηνύματα που δείχνουν αν ο λογαριασμός βρέθηκε ή όχι,
- για τους σκοπούς του demo, επιστρέφει True αν το account_info.value δεν είναι None, αλλιώς False

Επίσης ορίζεται η κλάση **SecureChannelWithBlockchain(SecureChannel)**, η οποία επεκτείνει το AEAD secure channel της 4ης φάσης με on-chain επαλήθευση. Κάθε στιγμιότυπο της SecureChannelWithBlockchain διατηρεί:

- όλα τα πεδία της SecureChannel (βλ. φάση 4),
- μια Solana διεύθυνση πορτοφολιού self.solana_address (string),
- ένα self.registry_client (instance της SolanaKeyRegistryClient).

Η μέθοδος **verify_peer_via_blockchain(peer_solana_address: str, peer_signing_pub_bytes: bytes) -> bool:**

- καταγράφει τη Solana διεύθυνση και το κλειδί υπογραφής του peer
- καλεί την self.registry_client.verify_key(peer_solana_address, peer_signing_pub_bytes)
- εμφανίζει είτε “blockchain verification SUCCESS” είτε “blockchain verification FAILED” και επιστρέφει το αντίστοιχο boolean.

Η μέθοδος **register_key_on_blockchain(wallet_keypair: Keypair):**

- διαβάζει τα bytes του δημοσίου κλειδιού υπογραφής (signing public key) του συμμετέχοντος από την self.participant.signing_public_bytes (όπως έχει υλοποιηθεί στη Φάση 4),
- καλεί την self.registry_client.register_key(wallet_keypair, signing_pub_bytes),
- καταγράφει την υπογραφή (signature) της επιστρεφόμενης συναλλαγής και τη Solana διεύθυνση.

```
    def register_key_on_blockchain(self, wallet_keypair):
        print(f"\n{self.name}: Registering signing key on Solana blockchain...")
        signing_pub_bytes = self.participant.signing_public_bytes
        tx_sig = self.registry_client.register_key(wallet_keypair, signing_pub_bytes)
        print(f"✅ {self.name}: Key registered on-chain")
        print(f"   Transaction: {tx_sig}")
        print(f"   Solana address: {self.solana_address}")
```

```
        print(f"   Signing key (hex): {signing_pub_bytes.hex()}")
```

Η συνάρτηση **demonstrate_blockchain_integration()** εκτελεί το πλήρες σενάριο της 5ης φάσης:

- Δημιουργεί έναν SolanaKeyRegistryClient για το Devnet.
- Ορίζει δύο mock διευθύνσεις, την alice_address και την bob_address.
- Δημιουργεί το alice_channel = SecureChannelWithBlockchain("Alice", alice_address, registry_client) και το bob_channel = SecureChannelWithBlockchain("Bob", bob_address, registry_client).
- Καλεί την alice_channel.participant.generate_keypairs() και την bob_channel.participant.generate_keypairs() για να προετοιμάσει τα τοπικά DH και τα κλειδιά υπογραφών (όπως στις φάσεις 3 και 4).
- Δημιουργεί mock πορτοφόλια Solana (mock_alice_wallet = Keypair(), mock_bob_wallet = Keypair()) και καλεί την register_key_on_blockchain(...) και στα δύο κανάλια, ώστε να «καταχωριστούν» τα κλειδιά υπογραφών τους στην αλυσίδα (on-chain)
- Η Alice στέλνει το αυθεντικοποιημένο DH δημόσιο κλειδί της στον Bob μέσω της alice_channel.get_public_keys(), και στη συνέχεια:
    - ο Bob καλεί πρώτα την bob_channel.verify_peer_via_blockchain(alice_address, alice_signing_pub)
    - και μόνο αν αυτό επιστρέψει True, καλεί την bob_channel.establish_channel(alice_dh_pub, alice_signing_pub, alice_sig) για να εκτελέσει την αυθεντικοποιημένη ανταλλαγή κλειδιών.
- Εκτελείται μία συμμετρική διαδικασία για τα κλειδιά του Bob προς την Alice, με την επαλήθευση να πραγματοποιείται μέσω της verify_peer_via_blockchain(...) πριν από κάθε establish_channel(...).
- Αφού εγκατασταθούν και οι δύο κατευθύνσεις, χρησιμοποιούνται οι μέθοδοι send_message() / receive_message() από την κλάση SecureChannel για την ανταλλαγή ενός μηνύματος και εμφανίζεται ότι η ασφαλής επικοινωνία έχει εδραιωθεί με την επαλήθευση μέσω blockchain.

5.4.2 Smart contract (programs/solana_key_registry/src/lib.rs)

Το on-chain τμήμα του συστήματος υλοποιείται ως Solana πρόγραμμα βασισμένο στο Anchor framework και περιέχεται στο αρχείο backend/phases/phase5_solana/solana_key_registry/programs/solana_key_registry/src/lib.rs. Ορίζει το KeyRecord και τις εντολές registerKey, updateKey και verifyKey, οι οποίες αποτυπώνονται στο IDL το οποίο φορτώνεται από τον Python client.

Η βασική δομή του λογαριασμού **KeyRecord**, είναι η εξής:

```
#[account]
pub struct KeyRecord {
    pub owner: Pubkey,          // wallet address
    pub public_key: [u8; 32], // Ed25519 public key
    pub bump: u8,               // PDA bump
}
```

όπου:

- owner είναι το Solana wallet Pubkey το οποίο και ελέγχει την εγγραφή
- public_key είναι το 32-byte κλειδί υπογραφής Ed25519 που συνδέεται με το συγκεκριμένο πορτοφόλι
- bump είναι το PDA bump seed που χρησιμοποιείται κατά την παραγωγή της διεύθυνσης του λογαριασμού

Οι κύριες εντολές του προγράμματος ακολουθούν το IDL που χρησιμοποιεί η **SolanaKeyRegistryClient._load_idl():**

- **registerKey**:
    - παράγει την PDA KeyRecord από τα seeds ["key_record", owner] και το program ID
    - δημιουργεί και αρχικοποιεί τον λογαριασμό KeyRecord σε αυτή την PDA
    - αποθηκεύει τα πεδία owner και το περιεχόμενο του publicKey
    - επιβάλλει ότι ο owner πρέπει να είναι ο signer της συναλλαγής, ώστε μόνο ο κάτοχος του wallet να μπορεί να καταχωρίσει κάποιο κλειδί για τη ίδια τη διεύθυνσή του.
- **updateKey**:

    - ελέγχει ότι ο owner που υπογράφει τη συναλλαγή ταυτίζεται με το πεδίο owner που είναι ήδη αποθηκευμένο στο KeyRecord,
    - ενημερώνει το πεδίο public_key με μια νέα τιμή newPublicKey, επιτρέποντας την αλλαγή των κλειδιών χωρίς αλλαγή ιδιοκτησίας.
- **verifyKey**:
    - διαβάζει τον λογαριασμό KeyRecord για το κάθε PDA
    - συγκρίνει τον αποθηκευμένο πίνακα public_key με το όρισμα publicKeyToVerify
    - επιστρέφει μια boolean τιμή που δείχνει αν ταιριάζουν, επιτρέποντας σε έναν off-chain client (όπως η SolanaKeyRegistryClient.verify_key(...)) να χρησιμοποιεί το μητρώο ως ένα αποκεντρωμένο μητρώο κλειδιών, στο οποίο οι εγγραφές έχουν τη δυνατότητα μόνο να προστίθενται και όχι να διαγράφονται ή να τροποποιούνται (append-only key directory).

5.4.3 Σενάρια Επιθέσεων στο Blockchain (phase6_blockchain_attack/blockchain_mitm_attack.py)

Η 6η φάση έχει υλοποιηθεί στο backend/phases/phase6_blockchain_attack/blockchain_mitm_attack.py και αναπαριστά επιθέσεις στο blockchain και πιο συγκεκριμένα απέναντι στο μητρώο κλειδιών που υλοποιήθηκε στην προηγούμενη φάση.

Για να πραγματοποιηθούν οι επιθέσεις αυτές, εισάγονται:

- η κλάση AuthenticatedParticipant από την 3η φάση
- το SolanaKeyRegistryClient από την 5η φάση
- τα στοιχεία X25519 και Ed25519, η HKDF και η InvalidSignature, όπως και στις προηγούμενες φάσεις

Η κλάση **BlockchainRegistry** παρέχει ένα προσομοιωμένο μητρώο κλειδιών το οποίο αντικατοπτρίζει τη συμπεριφορά του Solana smart contract:

- Αποθηκεύει ένα λεξικό self.registry: dict[str, bytes], το οποίο αντιστοιχίζει τις διευθύνσεις των πορτοφολιών σε δημόσια κλειδιά Ed25519.
- Η μέθοδος **register_key(wallet_address: str, wallet_keypair, ed25519_public_key: bytes) -> bool:**

    - υπολογίζει το wallet_pubkey = str(wallet_keypair.pubkey()),
    - απορρίπτει τη λειτουργία, αν το wallet_pubkey != wallet_address, καταγράφοντας την αδυναμία αντιστοίχισης της διεύθυνσης με το πορτοφόλι και επιστρέφοντας False
    - διαφορετικά, αποθηκεύει το ed25519_public_key στο wallet_address, καταγράφει την καταχώριση και επιστρέφει True.
    - Με τον τρόπο αυτό προσομοιώνεται η απαίτηση ότι μόνο ο κάτοχος του πορτοφολιού μπορεί να καταχωρίσει κάποιο κλειδί για τη δική του διεύθυνση
- Η μέθοδος **verify_key(wallet_address: str, ed25519_public_key: bytes) -> bool:**
    - αποτυγχάνει αν δεν υπάρχει κάποιο καταχωρισμένο κλειδί για το wallet_address
    - διαφορετικά, συγκρίνει το αποθηκευμένο κλειδί με το ed25519_public_key
    - αν τα κλειδιά ταυτίζονται, καταγράφεται ως επιτυχία, ενώ σε περίπτωση μη ταύτισης (με εμφάνιση των expected vs received hex prefixes) καταγράφεται ως αποτυχία και εμφανίζει προειδοποίηση για πιθανή απόπειρα επίθεσης MITM.

Η κλάση **BlockchainMallory** αναπαριστά τις επιθέσεις του Mallory απέναντι στο μητρώο του blockchain:

- Στην **__init__(...)** η κλάση:
    - δημιουργεί τα δικά της κλειδιά X25519 και Ed25519 (self.dh_priv, self.signing_priv, self.signing_pub_bytes)
    - ορίζει μια σταθερή διεύθυνση του πορτοφολιού του self.mallory_wallet_address = "Mallory1111111111111111111111111111111111"
    - δημιουργεί ένα mock ζεύγος κλειδιών για το πορτοφόλι self.mallory_wallet_keypair, του οποίου η μέθοδος pubkey() επιστρέφει τη συγκεκριμένη διεύθυνση
    - αποθηκεύει μια αναφορά στο κοινό instance του BlockchainRegistry.

Η κλαση BlockchainMallory υλοποιεί **τέσσερις στοχευμένες μεθόδους για τις επιθέσεις**, καθεμία από τις οποίες επιστρέφει True όταν η επίθεση αποτρέπεται:

1. **attack_1_register_alice_key_with_alice_address(alice_address: str, alice_signing_pub_bytes: bytes) -> bool**

- Ο Mallory καλεί:

```
success = self.registry.register_key(
    alice_address,
    self.mallory_wallet_keypair,  # Mallory's wallet (WRONG!)
    alice_signing_pub_bytes
)
```

- Επειδή το pubkey() του mock ζεύγους κλειδιών δεν ταιριάζει με το alice_address, η μέθοδος BlockchainRegistry.register_key(...) απορρίπτει τη λειτουργία
- Η μέθοδος καταγράφει ότι η επίθεση αποτράπηκε και επιστρέφει True

2. **attack_2_register_own_key_with_alice_address(alice_address: str) -> bool**
   - Ο Mallory προσπαθεί να καταχωρίσει το δικό του κλειδί Ed25519 στη διεύθυνση της Alice:

```
success = self.registry.register_key(
    alice_address,
    self.mallory_wallet_keypair,  # Mallory's wallet (WRONG!)
    self.signing_pub_bytes
)
```

   - Για τον ίδιο λόγο με την 1η επίθεση, το μητρώο απορρίπτει την καταχώριση λόγω μη ταύτισης της διεύθυνσης με το πορτοφόλι.
   - Η μέθοδος καταγράφει ότι και η 2η επίθεση αποτράπηκε και επιστρέφει True.

3. **attack_3_use_alice_key_with_own_address(alice_address: str, alice_signing_pub_bytes: bytes) -> bool**
   - Πρώτα, ο Mallory καταχωρεί το δικό του κλειδί για τη δική του διεύθυνση:

```
self.registry.register_key(
    self.mallory_wallet_address,
    self.mallory_wallet_keypair,
    self.signing_pub_bytes
)
```

   - Στη συνέχεια προσπαθεί να εκμεταλλευτεί το κλειδί της Alice που εκλάπη, στέλνοντας τα alice_signing_pub_bytes και ισχυρίζεται ότι ανήκει στην Alice.
   - Ο έλεγχος του Bob:

```
verification_result = self.registry.verify_key(alice_address, alice_signing_pub_bytes)
```

- Εφόσον το registry έχει αποθηκεύσει το γνήσιο κλειδί της Alice για την διεύθυνση alice_address, η επαλήθευση αποτυγχάνει, καθώς είτε το κλειδί, είτε η διεύθυνση δεν ταιριάζουν με ό,τι είναι καταχωρισμένο.
- Η μέθοδος καταγράφει ότι η on-chain επαλήθευση του Bob μπλόκαρε την επίθεση και επιστρέφει True όταν το verification_result είναι False.

4. **attack_4_register_fake_key_for_own_address() -> bool**
   - Ο Mallory καταχωρίζει το δικό του κλειδί για τη δική του διεύθυνση

```
        success = self.registry.register_key(
            self.mallory_wallet_address,
            self.mallory_wallet_keypair,
            self.signing_pub_bytes
        )
```

   - Στη συνέχεια, ο κώδικας προσομοιώνει τον έλεγχο που πραγματοποιεί ο Bob, ο οποίος επαληθεύει το κλειδί του Mallory έναντι της διεύθυνσης της Alice

```
            bob_verification = self.registry.verify_key(
                "Alice1111111111111111111111111111111111",  # Alice's address
                self.signing_pub_bytes  # Mallory's key
            )
```

   - Ο έλεγχος αυτός αποτυγχάνει, διότι ο Bob ελέγχει πάντα το μητρώο για τη διεύθυνση της Alice και όχι του Mallory. Η μέθοδος καταγράφει ότι η επίθεση αποτρέπεται. Η καταχώριση του Mallory είναι επιτυχής, αλλά δεν κατάφερε να υποδυθεί την Alice.
   - Η attack_4_register_fake_key_for_own_address() επιστρέφει True για να δηλώσει ότι ο στόχος της επίθεσης (impersonation) έχει μπλοκαριστεί.

Η συνάρτηση **_blockchain_mitm_attack()** ενορχηστρώνει όλο το σενάριο της 6ης φάσης:

- Αρχικοποιεί το registry = BlockchainRegistry().
- Δημιουργεί τις αυθεντικοποιημένες οντότητες alice = AuthenticatedParticipant("Alice") και bob = AuthenticatedParticipant("Bob") και καλεί τις alice.generate_keypairs() και bob.generate_keypairs(), όπως και στη 3η φάση.
- Ορίζει τις σταθερές διευθύνσεις:

```
    alice_address = "Alice1111111111111111111111111111111111"
    bob_address = "Bob1111111111111111111111111111111111111"
```

- Δημιουργεί τα mock ζεύγη κλειδιών των πορτοφολιών alice_wallet και bob_wallet, των οποίων η συνάρτηση pubkey() επιστρέφει τις παραπάνω διευθύνσεις, και καταχωρίζει τα κλειδιά υπογραφών τους στο μητρώο μέσω της registry.register_key(...).
- Δημιουργεί τον mallory = BlockchainMallory(registry) και καλεί:

```
attack1_prevented = mallory.attack_1_register_alice_key_with_alice_address(
    alice_address,
    alice.signing_public_bytes
)
attack2_prevented = mallory.attack_2_register_own_key_with_alice_address(
    alice_address
)
attack3_prevented = mallory.attack_3_use_alice_key_with_own_address(
    alice_address,
    alice.signing_public_bytes
)
attack4_prevented = mallory.attack_4_register_fake_key_for_own_address()
```

- Εμφανίζει μια σύνοψη, που για κάθε επίθεση δηλώνει αν είναι “PREVENTED” ή “SUCCEEDED”, μαζί με τη σχετική αιτιολόγηση.

6. Μαθήματα ασφαλείας, Συμπεράσματα και Μελλοντική Έρευνα

6.1 Συμπεράσματα για τον σχεδιασμό των Secure Channel

Η κλιμακωτή σχεδίαση του συστήματος μέσω των φάσεων δείχνει πως η σωστή κρυπτογραφία από μόνη της δεν αρκεί, για να υπάρχει ασφαλής επικοινωνία.

- **Ο μη αυθεντικοποιημένος Diffie-Hellman δεν είναι αρκετός**

  Στη 1η φάση , επιβεβαιώνεται πως το X25519 παράγει σωστά ένα κοινό μυστικό. Η 2η φάση όμως, δείχνει ότι σε ένα “εχθρικό” δίκτυο αυτό δεν αρκεί καθώς ένας επιτιθέμενος man-in-the-middle μπορεί να εγκαθιδρύσει δύο έγκυρα κοινά κλειδιά (Alice-Mallory και Bob-Mallory), ενώ ταυτόχρονα η Alice και ο Bob δεν θα μοιραστούν ποτέ ένα κοινό κλειδί μεταξύ τους [1] [14].
- **Η αυθεντικοποίηση πρέπει να "δένει” τις ταυτότητες με τα κλειδιά**

  Στη 3η φάση, οι υπογραφές Ed25519 εφαρμόζονται στα δημόσια κλειδιά DH και η ίδια λογική επίθεσης MITM που πέτυχε στη 2η φάση, τώρα αποτυγχάνει. Αυτό συμβαίνει,

καθώς τα κλειδιά που χρησιμοποιούνται στην ανταλλαγή DH, τώρα συνδέονται με τις ταυτότητες πριν γίνει αποδεκτό οποιοδήποτε κοινό κλειδί [2], [15], [17].

- **Η χρήση του AEAD ολοκληρώνει το secure channel**

  Η 4η φάση δείχνει πώς το ChaCha20-Poly1305 μετατρέπει ένα αυθεντικοποιημένο κοινό κλειδί σε ένα πλήρες secure channel, καθώς τα «τίμια» μηνύματα αποκρυπτογραφούνται στο αρχικό plaintext, ενώ ακόμη και η αλλαγή ενός μόνο bit στο ciphertext οδηγεί σε απόρριψη [11].

Συνολικά, η πορεία εξέλιξης του μη αυθεντικοποιημένο DH σε αυθεντικοποιημένο DH και και τελικά σε ένα πλήρες secure channel με AEAD, δίνει μια ουσιαστική εικόνα για το πώς χτίζονται τα σύγχρονα secure channels από βασικά κρυπτογραφικά στοιχεία.

6.2 Συμπεράσματα για την αυθεντικοποίηση μέσω Blockchain

Οι φάσεις 5 και 6 οι οποίες ασχολούνται με το blockchain μέρος της εργασίας, αναδεικνύουν τη διαφορά ανάμεσα στην "τοπική" (local configuration) εμπιστοσύνη και στην σύνδεση των ταυτοτήτων με μια εξωτερική πηγή εμπιστοσύνης, η οποία στην συγκεκριμένη περίπτωση είναι το blockchain.

- **Μετάβαση από την τοπική εμπιστοσύνη σε ένα εξωτερικό μητρώο κλειδιών.**

  Στις φάσεις 1 με 4, θεωρούμε πως τα κλειδιά που χρησιμοποιούνται είναι γνήσια. Η 5η φάση αντικαθιστά αυτή την παραδοχή με ένα μητρώο στο Solana, στο οποίο τα δημόσια κλειδιά αποθηκεύονται ως "program-derived" λογαριασμοί, οι οποίοι έχουν ως βάση τις διευθύνσεις των πορτοφολιών. Οι οντότητες που επικοινωνούν, πλέον δεν θεωρούνται αξιόπιστες απλώς και μόνο επειδή το πιστεύουμε, αλλά επειδή το blockchain δεσμεύει μια συγκεκριμένη διεύθυνση σε ένα δημόσιο κλειδί των 32-byte [16], [20].

- **Το πώς έχει "στηθεί" το μητρώο είναι εξίσου σημαντικό με την ίδια την κρυπτογραφία**

  Η 6η φάση δείχνει πως σχετικά απλοί κανόνες, όπως το γεγονός ότι μόνο ο κάτοχος ενός πορτοφολιού θα μπορεί να καταχωρίσει ή να ενημερώσει το KeyRecord του, και ότι αυτοί που είναι υπεύθυνοι για τις επαληθεύσεις ρωτούν πάντα το μητρώο με τη διεύθυνση του προοριζόμενου μέρους, αρκούν για να μπλοκάρουν τέσσερις απλές απόπειρες πλαστοπροσωπίας. Τα οφέλη που αποκομίζουμε απο τη χρήση του blockchain δεν προκύπτουν από το blockchain «από μόνο του», αλλά από τη συνεπή

επιβολή της δέσμευσης διεύθυνσης-κλειδιού και τον έλεγχο αυτής της δέσμευσης σε κάθε προσπάθεια ταυτοποίησης [13] [6].

Το βασικό δίδαγμα είναι ότι ένα blockchain μπορεί να λειτουργήσει ως μία ουσιαστική "άγκυρα" εμπιστοσύνης, μόνο εάν κάθε οντότητα ελέγχεται στο μητρώο.

6.3 Μελλοντική Έρευνα

Για την συνέχιση της παρούσας εργασίας, προτείνονται οι εξής κατευθύνσεις:

- **Ενισχυμένη διαχείριση κλειδιών:**
  Υλοποίηση ενός πλήρους κύκλου ζωής των κλειδιών στο μητρώο (ανάκληση, λήξη, περιστροφή και ανάκτηση) και ενημέρωση των ήδη ενεργών καναλιών κάθε φορά που συμβαίνει μια τέτοια αλλαγή [21], [22].
- **Πραγματική υλοποίηση on-chain:**
  Μετάβαση από το προσομοιωμένο μητρώο σε ένα πλήρως ανεπτυγμένο Solana smart contract το οποίο θα γίνει "deploy" στο Devnet/Mainnet.
- **Παραλλαγές πρωτοκόλλου:**
  Πειραματισμός με εναλλακτικά AEAD σχήματα ή ciphersuites και σύγκριση της σύνδεσης των ταυτοτήτων στο Solana ή και σε άλλα blockchains.
- **Περισσότερα μοντέλα αντιπάλων**
  Επέκταση των επιθέσεων, ώστε να περιλαμβάνουν παραβιασμένα πορτοφόλια, κακόβουλα RPC endpoints και μελέτη του πώς αυτά επηρεάζουν την εμπιστοσύνη που υπάρχει στο μητρώο και στα secure channels [23].

Η συνέχιση της εργασίας σε αυτές τις κατευθύνσεις, θα μετέτρεπε τον ρόλο του υλοποιημένου συστήματος από ένα εκπαιδευτικό εργαλείο σε ένα ευρύτερο πειραματικό πλαίσιο για secure channels με αποκεντρωμένη αυθεντικοποίηση, η οποία θα είναι βασισμένη σε blockchain.

Βιβλιογραφία

[1] W. Shen, Y. Cheng, B. Yin, K. Liu, and X. Cao, “Diffie-Hellman in the Air: A Link Layer Approach for In-Band Wireless Pairing,” *IEEE Trans. Veh. Technol.*, vol. 70, pp. 11894–11907, 2019, doi: 10.1109/tvt.2021.3116619.

[2] B. Dowling, M. Fischlin, F. Günther, and D. Stebila, “A Cryptographic Analysis of the TLS 1.3 Handshake Protocol,” *J. Cryptol.*, vol. 34, 2020, doi: 10.1007/s00145-021-09384-1.

[3] R.Radzali *et al.*, “Analysis of Trust Models in Public Key Infrastructure: A Systematic Literature Review of Interoperability Challenges,” *Malays. J. Sci. Health Technol.*, 2025, doi: 10.33102/mjosht.v11i1.465.

[4] K. Tange, M. D. Donno, X. Fafoutis, and N. Dragoni, “A Systematic Survey of Industrial Internet of Things Security: Requirements and Fog Computing Opportunities,” *IEEE Commun. Surv. Tutor.*, vol. 22, pp. 2489–2520, 2020, doi: 10.1109/comst.2020.3011208.

[5] Y. Li, Y. Yu, C. Lou, N. Guizani, and L. Wang, “Decentralized Public Key Infrastructures atop Blockchain,” *IEEE Netw.*, vol. 34, pp. 133–139, 2020, doi: 10.1109/mnet.011.2000085.

[6] G. A. Pierro and R. Tonelli, “Can Solana be the Solution to the Blockchain Scalability Problem?,” *2022 IEEE Int. Conf. Softw. Anal. Evol. Reengineering SANER*, pp. 1219–1226, 2022, doi: 10.1109/saner53432.2022.00144.

[7] A. Yakovenko, “Solana : A new architecture for a high performance blockchain v 0 . 8,” 2018.

[8] A. Langley, M. Hamburg, and S. Turner, “Elliptic Curves for Security,” *RFC*, vol. 7748, pp. 1–22, 2016, doi: 10.17487/rfc7748.

[9] D. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” *J. Cryptogr. Eng.*, vol. 2, pp. 77–89, 2011, doi: 10.1007/s13389-012-0027-1.

[10] H. Krawczyk, “Cryptographic Extraction and Key Derivation: The HKDF Scheme,” pp. 631–648, 2010, doi: 10.1007/978-3-642-14623-7_34.

[11] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” *RFC*, vol. 8439, pp. 1–46, 2018, doi: 10.17487/rfc7539.

[12] D. McGrew, “An Interface and Algorithms for Authenticated Encryption,” *RFC*, vol. 5116, pp. 1–22, 2008, doi: 10.17487/rfc5116.

[13] G. B. Blocks, “An Introduction to Solana,” *No Dec.*, 2021.

[14] A. S. Khader and D. Lai, “Preventing man-in-the-middle attack in Diffie-Hellman key exchange protocol,” *2015 22nd Int. Conf. Telecommun. ICT*, pp. 204–208, 2015, doi: 10.1109/ict.2015.7124683.

[15] H. Krawczyk, “SIGMA: The ‘SIGn-and-MAc’ Approach to Authenticated Diffie-Hellman and Its Use in the IKE-Protocols,” pp. 400–425, 2003, doi: 10.1007/978-3-540-45146-4_24.

[16] M. Toorani and C. Gehrmann, “A decentralized dynamic PKI based on blockchain,” *Proc. 36th Annu. ACM Symp. Appl. Comput.*, 2020, doi: 10.1145/3412841.3442038.

[17] J. Pan, C. Qian, and M. Ringerud, “Signed (Group) Diffie–Hellman Key Exchange with Tight Security,” *J. Cryptol.*, vol. 35, 2022, doi: 10.1007/s00145-022-09438-y.

[18] J. Brendel, C. J. F. Cremers, D. Jackson, and M. Zhao, “The Provable Security of Ed25519: Theory and Practice,” *2021 IEEE Symp. Secur. Priv. SP*, pp. 1659–1676, 2021, doi: 10.1109/sp40001.2021.00042.

[19] M. Bellare, R. Ng, and B. Tackmann, “Nonces are Noticed: AEAD Revisited,” pp. 235–265, 2019, doi: 10.1007/978-3-030-26948-7_9.
[20] C. Fromknecht, D. Velicanu, and S. Yakoubov, “A Decentralized Public Key Infrastructure with Identity Retention,” *IACR Cryptol EPrint Arch*, vol. 2014, p. 803, 2014.
[21] C.-S. Park and H.-M. Nam, “A New Approach to Constructing Decentralized Identifier for Secure and Flexible Key Rotation,” *IEEE Internet Things J.*, vol. 9, pp. 10610–10624, 2022, doi: 10.1109/jiot.2021.3121722.
[22] B. Kim, Y.-S. Cho, S.-H. Kim, H. Kim, and S. S. Woo, “A Security Analysis of Blockchain-Based Did Services,” *IEEE Access*, vol. 9, pp. 22894–22913, 2021, doi: 10.1109/access.2021.3054887.
[23] N. Naik, P. Grace, P. Jenkins, S. Naik, and J. Song, “An evaluation of potential attack surfaces based on attack tree modelling and risk matrix applied to self-sovereign identity,” *Comput Secur*, vol. 120, p. 102808, 2022, doi: 10.1016/j.cose.2022.102808.

Λέξεις Κλειδιά

- **Alice**: Η οντότητα “Alice” στο πρωτόκολλο, που εκτελεί το handshake, χρησιμοποιεί τα κλειδιά και στέλνει μηνύματα προς τον Bob.
- **Anchor Framework**: Το framework (Anchor) που χρησιμοποιείται για την υλοποίηση του Solana smart contract.
- **Bob**: Η οντότητα “Bob” είναι ο συνομιλητής της Alice, ο οποίος συμμετέχει στο handshake, παράγει ένα κοινό μυστικό και αποκρυπτογραφεί τα μηνύματα.
- **Blockchain**: Ένα αποκεντρωμένο σύστημα, το οποίο καταγράφει συναλλαγές και δεδομένα σε μία αλυσίδα από blocks.
- **Ciphertext**: Το κρυπτογραφημένο μήνυμα που παράγεται από το AEAD.
- **Component**: Ένα διακριτό μέρος του συστήματος (π.χ. UI, backend, phase modules, blockchain integration).
- **Decentralized**: Μοντέλο στο οποίο η εμπιστοσύνη δεν στηρίζεται σε κάποια κεντρική αρχή, αλλά σε ένα δημόσιο δίκτυο και σε on-chain καταχωρήσεις (πχ. το μητρώο στο Solana).
- **Deploy On Devnet/Mainnet**: Η διαδικασία δημοσίευσης (deploy) του smart contract στο αντίστοιχο περιβάλλον (devnet ή mainnet), ώστε ο client να αλληλεπιδρά με το πραγματικό on-chain μητρώο.
- **Handshake**: Η αρχική “γνωριμία” δύο μερών (Alice-Bob), στην οποία ανταλλάσσουν τις απαραίτητες πληροφορίες/κλειδιά για να συμφωνήσουν σε ένα κοινό μυστικό και να ξεκινήσουν την ασφαλή επικοινωνία.
- **Instance**: Ένα συγκεκριμένο “στιγμιότυπο” ενός αντικειμένου ή μιας κλάσης.
- **Mallory**: Μία οντότητα η οποία προσομοιώνει έναν ενεργό αντίπαλο.
- **Module**: Ένα “ξεχωριστό κομμάτι” του κώδικα το οποίο υλοποιεί μια συγκεκριμένη λειτουργία ή φάση του συστήματος.
- **Mock Διεύθυνση/Mock Πορτοφόλι**: “Ψεύτικη” διεύθυνση ή πορτοφόλι που χρησιμοποιείται χωρίς να είναι ένα πραγματικό πορτοφόλι στο blockchain.
- **Nonce**: Μία μοναδική τιμή (12 bytes στο ChaCha20-Poly1305) που συνοδεύει κάθε AEAD κρυπτογράφηση, ώστε δύο ίδια μηνύματα να μην βγάζουν ποτέ το ίδιο αποτέλεσμα.
- **PKI - Public Key Infrastructure**: Το κλασικό σύστημα στο οποίο μια “αρχή” (CA) εκδίδει πιστοποιητικά τα οποία αποδεικνύουν ότι ένα δημόσιο κλειδί ανήκει σε μία συγκεκριμένη οντότητα.
- **Raise Error**: Ο κώδικας, όταν εντοπίζει κάποιο λάθος, σταματά και εμφανίζει σφάλμα.

- **Registry**: Ένα μητρώο που αποθηκεύει στο blockchain ποιο δημόσιο κλειδί Ed25519 αντιστοιχεί σε κάθε Solana διεύθυνση, ώστε όλοι να μπορούν να το ελέγχουν χωρίς την ανάγη ύπαρξης κάποιας κεντρικής αρχής.
- **Secure Channel**: Ένα ασφαλές κανάλι επικοινωνίας στο οποίο αρχικά δύο μέρη συμφωνούν σε κοινό κλειδί, ώστε να μπορούν να ανταλλάσσουν κρυπτογραφημένα μηνύματα.
- **Smart Contract**: Το Solana πρόγραμμα (γραμμένο σε Anchor/Rust) το οποίο υλοποιεί το μητρώο (KeyRecord) και άλλους κανόνες.
- **Solana**: Το blockchain που χρησιμοποιείται στο σύστημα για το αποκεντρωμένο μητρώο κλειδιών και για την on-chain επαλήθευση των ταυτοτήτων.
- **Solana Client**: Το Python κομμάτι του συστήματος, το οποίο συνδέεται με το Solana δίκτυο.
- **Tampering**: Η σκόπιμη αλλοίωση δεδομένων (πχ. αλλοίωση bit σε ciphertext), για να αποδειχθεί ότι το AEAD εντοπίζει την αλλοίωση και απορρίπτει το μήνυμα.
- **TLS**: Το πρωτόκολλο που χρησιμοποιείται στο internet (π.χ. στο HTTPS) για να δημιουργεί ασφαλή σύνδεση ανάμεσα σε client και server, ταυτόχρονα κρυπτογραφώντας τα δεδομένα και πιστοποιώντας την ταυτότητα του server.
- **UI:** Το γραφικό περιβάλλον του συστήματος, στο οποίο ο χρήστης μπορεί να τρέξει κάθε φάση και να βλέπει τα αποτελέσματα και τα logs της υλοποίησης.

Λίστα εικόνων

1. Εικόνα 2.1: Αρχιτεκτονική υψηλού επιπέδου (frontend, Flask backend, modules των φάσεων, πρόγραμμα Solana).
2. Εικόνα 4.1: Το UI της πρώτης φάσης, με τα δημόσια κλειδιά της Alice και του Bob και ταυτιζόμενα συμμετρικά κλειδιά.
3. Εικόνα 4.2: Στιγμιότυπο μέρους της 2ης φάσης στο UI, η οποία απεικονίζει την επίθεση MITM και τη μη ταύτιση των κλειδιών που παράγονται από την Alice και τον Bob.
4. Εικόνα 4.3: Στιγμιότυπο του UI για τη 3η φάση.
5. Εικόνα 4.4: Στιγμιότυπο του UI για τη 4η φάση.
6. Εικόνα 4.5: Στιγμιότυπο του UI για τη φάση 5, με επιτυχημένη καταχώριση και επαλήθευση των κλειδιών της Alice και του Bob στο Solana blockchain.
7. Εικόνα 4.6: Στιγμιότυπο του UI για την 6η φάση.

Λίστα Πινάκων

1. Πίνακας 2.1: Βασικά στοιχεία της κρυπτογραφίας και του blockchain και ο ρόλος τους στο σύστημα.
2. Πίνακας 3.1: Κύρια στοιχεία (assets) και ιδιότητες ασφάλειας
3. Πίνακας 3.2: Τύποι αντιπάλων και οι δυνατότητές τους
4. Πίνακας 4.1: Σύνοψη των φάσεων και παρουσίαση του βασικού «μαθήματος ασφάλειας» που μεταδίδει καθεμία από αυτές.

Τοποθεσία Κώδικα και Οδηγίες Εμφάνισης του UI

Ο πλήρης κώδικας του συστήματος και οι οδηγίες διατίθενται σε repository στο GitHub (link).